

SAPIENZA
UNIVERSITÀ DI ROMA

Combinatorial Optimization

Volume 1 — Combinatorial optimization problems

Chapters 1–7 of the module «Combinatorial Optimization»: combinatorial optimization problems. Definitions, results with proofs and worked examples; exercises with complete solutions are in Exercises 1.

Fabio Furini

fabiofurini.github.io

Department of Computer, Control and Management Engineering — Sapienza University of Rome

Online version and downloadable material:
fabiofurini.github.io/combinatorial-optimization/
github.com/fabiofurini/combinatorial-optimization

Academic year 2026–2027

Contents

1 Combinatorial optimization problems **3**

1.1 Combinatorial optimization problems 4
1.2 The six problems of the course 5
1.3 Counting the solutions 9
1.4 Combinatorial explosion and brute force 11
1.5 Primal bounds, dual bounds and gap 14
1.6 A glimpse of computational complexity 17
1.7 Roadmap of the course 18
1.8 Typical mistakes 20
1.9 In summary 20
1.10 Exercises 21

2 Knapsack problems **25**

2.1 The knapsack problem 26
2.2 The subset sum problem 28
2.3 The bounded knapsack problem 30
2.4 The unbounded knapsack problem 32
2.5 The multidimensional knapsack problem 34
2.6 The multiple knapsack problem 36
2.7 The multiple-choice knapsack problem 38
2.8 The knapsack problem with setups 40
2.9 The variants at a glance 42
2.10 Remarks 42
2.11 Typical mistakes 43
2.12 In summary 43

3 Bin packing and cutting stock **44**

3.1 The bin packing problem 45
3.2 The cutting stock problem 48
3.3 Remarks 50
3.4 Typical mistakes 50
3.5 In summary 51

4 Vertex coloring **52**
4.1 A short review of graphs . . . 53
4.2 The vertex coloring problem . . . 54
4.3 Remarks . . . 58
4.4 Typical mistakes . . . 58
4.5 In summary . . . 58

5 The travelling salesman problem **60**
5.1 The travelling salesman problem . . . 61
5.2 Variants: metric and asymmetric TSP . . . 63
5.3 Counting the tours . . . 63
5.4 A first tour by hand . . . 64
5.5 A combinatorial lower bound . . . 65
5.6 Remarks . . . 66
5.7 Typical mistakes . . . 66
5.8 In summary . . . 66

6 Parallel machine scheduling **68**
6.1 Scheduling on identical parallel machines . . . 69
6.2 The three-field notation . . . 71
6.3 Counting the solutions . . . 71
6.4 A first schedule by hand . . . 72
6.5 A combinatorial lower bound . . . 72
6.6 Relation with the bin packing problem . . . 73
6.7 Remarks . . . 74
6.8 Typical mistakes . . . 74
6.9 In summary . . . 74

7 Capacitated lot sizing **76**
7.1 The capacitated lot sizing problem . . . 76
7.2 Variants . . . 78
7.3 The reference instance . . . 79
7.4 Counting the solutions . . . 79
7.5 A first plan by hand . . . 80
7.6 A combinatorial lower bound . . . 81
7.7 Remarks . . . 81
7.8 Typical mistakes . . . 82
7.9 In summary . . . 82

Chapter 1

Combinatorial optimization problems

Prerequisites

- From the *Preliminaries*: optimization problems (feasible region, objective function, optimal solution and optimal value), binomial coefficients and partitions of a finite set.
- No knowledge of integer programming is needed: the bounds of this chapter come from combinatorial arguments.

Objectives

At the end of this chapter you will be able to:

- recognize a combinatorial optimization problem and the combinatorial object behind its solutions (a subset, an ordering, a partition, an assignment);
- state in words the six problems of the course: knapsack, bin packing, vertex coloring, travelling salesman, scheduling on parallel machines and capacitated lot sizing;
- count the solutions of these problems (2^n subsets, $(n-1)!/2$ tours, Bell numbers for the partitions) and estimate the computing time of a brute force enumeration;
- define primal bounds, dual bounds and the gap, and use them to certify that a solution is optimal;
- place a problem in the classes P, NP and NP-hard, and tell a pseudo-polynomial algorithm from a polynomial one.

1.1 Combinatorial optimization problems

Definition 1.1: combinatorial optimization problem

Given:

1. a finite set $\mathscr{U} = \{1, 2, \ldots, n\}$, called the **ground set**,
2. a family $\mathcal{F} \subseteq 2^{\mathscr{U}}$ of subsets of $\mathscr{U}$, called the **feasible solutions**,
3. an **objective function** $f : \mathcal{F} \to \mathbb{R}$,

the associated **combinatorial optimization problem** asks to find a feasible solution $\tilde{S} \in \mathcal{F}$ such that

$$f(\tilde{S}) \geq f(S), \qquad \forall S \in \mathcal{F} \qquad \text{(maximization form)},$$

or such that $f(\tilde{S}) \leq f(S)$ for all $S \in \mathcal{F}$ (minimization form).

- The feasible region $\mathcal{F}$ is a **finite** set, since $|2^{\mathscr{U}}| = 2^n$. Accordingly, an optimal solution always exists whenever $\mathcal{F} \neq \emptyset$.
- In most problems the objective function is **linear** (additive): each element $j \in \mathscr{U}$ has a weight c_j and
$$f(S) = \sum_{j \in S} c_j.$$
This is the case of the knapsack problem (profits) and of the travelling salesman problem (distances). Some problems have a different objective, for example the **makespan** of the scheduling problem of Section 1.2, which is a maximum and not a sum.
- A **problem** is a general question with symbolic data; an **instance** I of the problem is obtained by fixing the numbers. We denote by $z(I)$ the optimal value of the instance I.

Combinatorial optimization is the study of optimization problems on **discrete structures** and **combinatorial objects**, such as graphs, subsets, permutations, sequences, partitions, coverings and packings.

- The **main characteristics** of combinatorial optimization problems are:
  1. the set of feasible solutions is **finite**;
  2. the number of feasible solutions is typically **enormous**: it grows exponentially (or faster) with the size of the instance;
  3. finding an optimal solution can be **computationally very difficult** (Section 1.6);
  4. specialized algorithms are needed to explore the solution space without enumerating it.

1.1.1 Four kinds of discrete decisions

- The discrete decisions met in practice fall into a few families, each one associated with a combinatorial object:

Decision	Examples	Combinatorial object
IN/OUT (inclusion)	select projects to fund with a limited budget; choose the products of a portfolio	a **subset**
ordering, sequencing	order in which jobs are processed; order of visit of the customers	a **permutation**
partitioning, packing	split items among bins; group exams into days	a **partition**
assignment	assign jobs to machines, students to projects	a **function** (a partition with labelled classes)

- A fifth family, **counting** decisions (how many units to produce, how many operators in a shift), leads to integer quantities, modelled by integer variables (module *Integer Linear Programming*, Chapter 1 «Integer linear programming»).

1.2 The six problems of the course

- The course is built around **six problems**. Each one is defined here in words, with a small instance; the formal definitions and the variants are in Chapters OC 2–7.
- The same problems return in every part of the course, solved each time with a different method; the reference instances of the knapsack, bin packing and coloring problems are introduced in Chapters OC 2–4.

1.2.1 Knapsack problem

Definition 1.2: Knapsack Problem

Given n items with profits p_j and weights w_j, and a capacity c, the **knapsack problem** (KP) asks to select a **subset** of items of total weight at most c and maximum total profit.

Example 1.1: a small knapsack instance

We consider the instance with $n = 4$ items and capacity $c = 9$:

item j	1	2	3	4
profit p_j	10	7	6	4
weight w_j	5	4	3	3

The subset $\{1, 3\}$ has weight $5 + 3 = 8 \leq 9$ and profit $10 + 6 = 16$; the subset $\{1, 2\}$ has weight $5 + 4 = 9 \leq 9$ and profit $10 + 7 = 17$. We call this instance Example 1.1.

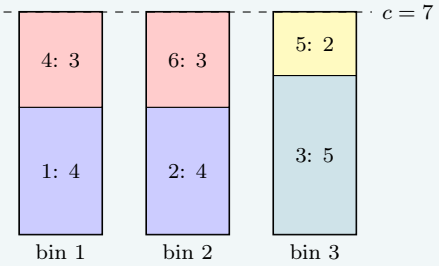
1.2.2 Bin packing problem

Definition 1.3: Bin Packing Problem

Given n items with weights w_j and identical bins of capacity c, the **bin packing problem** (BPP) asks to **partition** the items into the minimum number of bins, so that the total weight in each bin is at most c.

Example 1.2: a small bin packing instance

We consider $n = 6$ items with weights $\boldsymbol{w} = (4, 4, 5, 3, 2, 3)$ and capacity $c = 7$, and we call this instance Example 1.2. The partition $\{1, 4\}, \{2, 6\}, \{3, 5\}$ uses 3 bins, each one filled exactly:



1.2.3 Vertex coloring problem

Definition 1.4: Vertex Coloring Problem

Given a graph $G = (V, E)$, the **vertex coloring problem** (VCP) asks to assign a color to each vertex so that adjacent vertices have different colors, using the minimum number of colors. Equivalently, it asks to **partition** V into the minimum number of **stable sets** (sets of pairwise non-adjacent vertices). The minimum is the **chromatic number** $\chi(G)$.

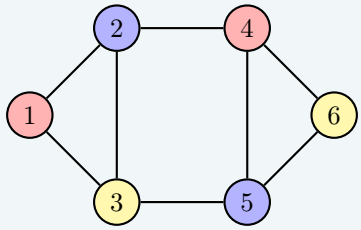
- In Example 1.3 the vertices are exams, the edges join the exams with students in common, and the colors are the days of the exam session.

Example 1.3: an exam calendar

We consider the exams 1 Calculus, 2 Linear algebra, 3 Physics, 4 Programming, 5 Statistics and 6 Operations research, with the conflicts

$$E = \{\{1,2\},\{1,3\},\{2,3\},\{2,4\},\{3,5\},\{4,5\},\{4,6\},\{5,6\}\}.$$

The coloring on the right uses 3 days: day 1 for $\{1,4\}$, day 2 for $\{2,5\}$, day 3 for $\{3,6\}$. Fewer days are impossible, since the exams $1,2,3$ are pairwise in conflict.



We call this instance Example 1.3.

1.2.4 Travelling salesman problem

Definition 1.5: Travelling Salesman Problem

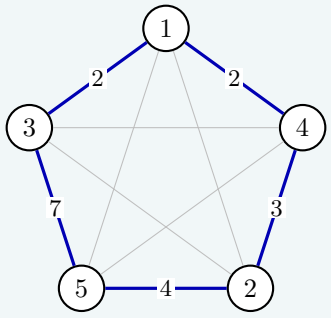
Given n cities and the distance d_{ij} between each pair of cities, the **travelling salesman problem** (TSP) asks to find a **tour**, that is a cyclic **ordering** of the cities visiting each city exactly once, of minimum total length.

Example 1.4: a small TSP instance

We consider $n = 5$ cities with the symmetric distances

d_{ij}	1	2	3	4	5
1	—	5	2	2	9
2	5	—	4	3	4
3	2	4	—	4	7
4	2	3	4	—	7
5	9	4	7	7	—

The tour $(1,3,5,2,4)$, in bold, has length $2+7+4+3+2=18$.



We call this instance Example 1.4.

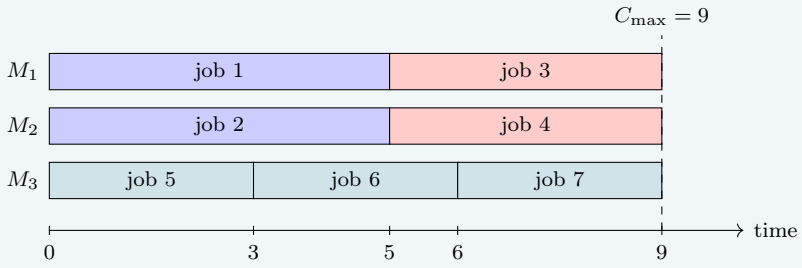
1.2.5 Scheduling on identical parallel machines

Definition 1.6: Scheduling on identical parallel machines

Given n jobs with processing times t_j and m identical machines, the problem $P\|C_{\max}$ asks to **assign** each job to a machine so as to minimize the **makespan** $C_{\max}$, that is the largest load L_h of a machine.

Example 1.5: a small $P\|C_{\max}$ instance

We consider $n = 7$ jobs with $\boldsymbol{t} = (5, 5, 4, 4, 3, 3, 3)$ and $m = 3$ machines, and we call this instance Example 1.5. The assignment $J_1 = \{1, 3\}$, $J_2 = \{2, 4\}$, $J_3 = \{5, 6, 7\}$ gives the loads $L_1 = 5 + 4 = 9$, $L_2 = 5 + 4 = 9$, $L_3 = 3 + 3 + 3 = 9$, hence $C_{\max} = 9$:



1.2.6 Capacitated lot sizing

Definition 1.7: Capacitated Lot Sizing Problem

Given m periods and, for each period, a demand, a production capacity, a setup cost (a fixed cost paid when the line produces), a unit production cost and a unit holding cost for the stock, the **capacitated lot sizing problem** (CLS) asks to choose the periods with a setup and the production quantities, so that the demand of every period is satisfied, the capacities are respected and the total cost (setup, production and holding costs) is minimum.

- The combinatorial part of the decision is the **subset** of periods with a setup: with m periods there are 2^m possible choices. Once the setups are fixed, the quantities are found by linear programming.

Problem	A solution is
knapsack (KP)	a subset of items
bin packing (BPP)	a partition of the items into bins
vertex coloring (VCP)	a partition into stable sets
TSP	a cyclic ordering of the cities
$P\|\|C_{\max}$	an assignment of jobs to machines
lot sizing (CLS)	setup periods and quantities

1.3 Counting the solutions

- Before solving a problem we ask **how many** candidate solutions it has. The answer depends only on the combinatorial object: subsets, orderings or partitions.

1.3.1 Subsets

Proposition 1.8

A set with n elements has 2^n subsets.

Proof. A subset $S \subseteq \{1, 2, \dots, n\}$ is identified by the binary vector $\boldsymbol{x} \in \{0,1\}^n$ with $x_j = 1$ if and only if $j \in S$. Each of the n components takes 2 values independently, hence there are $2 \cdot 2 \cdots 2 = 2^n$ vectors. Equivalently, the number of subsets with k elements is $\binom{n}{k}$, and we have:

$$\sum_{k=0}^{n} \binom{n}{k} = 2^n.$$

□

- Geometrically, the 2^n binary vectors are the vertices of the unit hypercube $[0,1]^n$: the square for $n = 2$ (4 vertices) and the cube for $n = 3$ (8 vertices).
- The candidate solutions of the knapsack problem with n items and the setup choices of the lot sizing problem with m periods are subsets: 2^n and 2^m of them, respectively.

1.3.2 Orderings and tours

- The n elements of a set can be ordered in $n! = n \cdot (n-1) \cdots 2 \cdot 1$ ways (**permutations**).

Proposition 1.9

For $n \geq 3$ cities with symmetric distances, the number of distinct tours is

$$\frac{(n-1)!}{2}.$$

Proof. A tour is a cyclic sequence of the n cities. Since the starting city does not change the tour, we can always start from city 1: the remaining $n-1$ cities can be ordered in $(n-1)!$ ways. Each tour is obtained twice, once for each direction of travel, for example $(1,3,5,2,4)$ and $(1,4,2,5,3)$, and both directions have the same length when the distances are symmetric. Hence the number of distinct tours is $(n-1)!/2$. □

Example 1.6: counting the tours

For the TSP of Example 1.4 we have $(5-1)!/2 = 24/2 = 12$ tours. Their lengths, computed by enumeration, are:

tour	L	tour	L	tour	L
$(1,3,5,2,4)$	**18**	$(1,3,4,2,5)$	22	$(1,2,3,5,4)$	25
$(1,3,2,5,4)$	19	$(1,4,3,2,5)$	23	$(1,3,2,4,5)$	25
$(1,2,5,3,4)$	22	$(1,2,4,5,3)$	24	$(1,4,2,3,5)$	25
$(1,2,5,4,3)$	22			$(1,2,4,3,5)$	28
				$(1,2,3,4,5)$	29

The tour $(1,3,5,2,4)$ is the unique optimal tour, and $z(I) = 18$.

1.3.3 Partitions and Bell numbers

- A **partition** of a finite set A is a collection of non-empty, pairwise disjoint subsets of A whose union is A. The solutions of the bin packing problem and of the vertex coloring problem are partitions (of the items and of the vertices).

- The number of partitions of a set with n elements is the **Bell number** $B(n)$, defined by the recursion:

$$B(0) = 1, \qquad B(n) = \sum_{k=0}^{n-1} \binom{n-1}{k} B(k), \quad \forall n \geq 1. \tag{1.1}$$

  The recursion counts the partitions by the block that contains element n: if this block has $n-1-k$ further elements, they can be chosen in $\binom{n-1}{n-1-k} = \binom{n-1}{k}$ ways, and the remaining k elements are partitioned in $B(k)$ ways.

- The first Bell numbers are:

n	0	1	2	3	4	5	6	7	8	9	10
$B(n)$	1	1	2	5	15	52	203	877	4140	21147	115975

- An **assignment** of n jobs to m machines is a function from jobs to machines: there are m^n of them. Since the machines of $P\|C_{\max}$ are identical, two assignments that differ only by a renaming of the machines are the same schedule; the distinct schedules are the partitions of the jobs into at most m blocks.

Example 1.7: how many candidate solutions

For these instances, the number of candidates and of feasible solutions, computed by enumeration, are:

instance	candidates	number	feasible
KP, Example 1.1	subsets of 4 items	$2^4 = 16$	11
BPP, Example 1.2	partitions of 6 items	$B(6) = 203$	30
VCP, Example 1.3	partitions of 6 vertices	$B(6) = 203$	22
TSP, Example 1.4	tours of 5 cities	$4!/2 = 12$	12
$P\|C_{\max}$, Example 1.5	assignments of 7 jobs	$3^7 = 2187$	2187
	partitions into at most 3 blocks	$1 + 63 + 301 = 365$	365

For the bin packing instance, a partition is feasible when every block weighs at most 7; for the coloring instance, when every block is a stable set. The 365 schedules of $P\|C_{\max}$ are the partitions of 7 jobs into 1, 2 or 3 blocks (1, 63 and 301 of them).

1.4 Combinatorial explosion and brute force

- Since the feasible region is finite, a combinatorial optimization problem can always be solved by **complete enumeration**: we list all candidate solutions, discard the infeasible ones and keep the best. This algorithm is called **brute force**.

Algorithm

Input: An instance of a combinatorial optimization problem in maximization form.

Output: An optimal solution and the optimal value.

Step 0: **Initialization**. Set $\bar{S} \leftarrow$ none and LB $\leftarrow -\infty$.

Step 1: **Enumeration**. For each candidate solution S (subset, tour, partition):

(a) if S is not feasible, discard it;

(b) if S is feasible and $f(S) >$ LB, set $\bar{S} \leftarrow S$ and LB $\leftarrow f(S)$.

Step 2: **Output**. Return $\tilde{S} = \bar{S}$ and $z(I) =$ LB. `STOP`

Example 1.8: brute force on the knapsack instance

For the knapsack instance of Example 1.1 the $2^4 = 16$ subsets are:

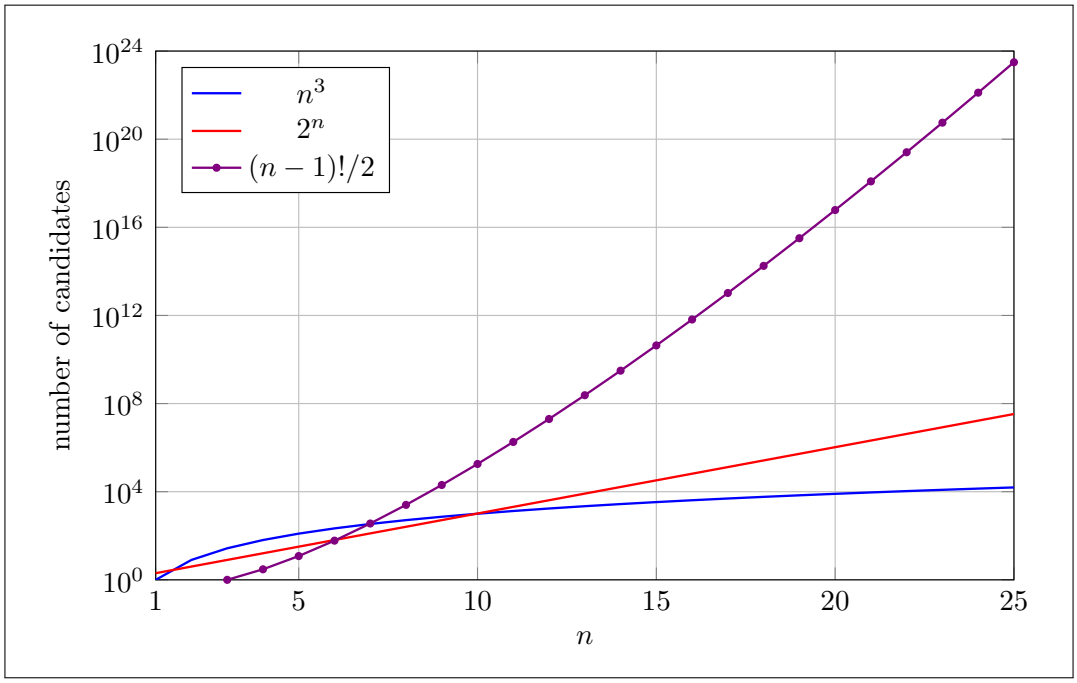
S	weight	profit	feasible	S	weight	profit	feasible
$\emptyset$	0	0	yes	$\{2,3\}$	7	13	yes
$\{1\}$	5	10	yes	$\{2,4\}$	7	11	yes
$\{2\}$	4	7	yes	$\{3,4\}$	6	10	yes
$\{3\}$	3	6	yes	$\{1,2,3\}$	12	23	no
$\{4\}$	3	4	yes	$\{1,2,4\}$	12	21	no
$\{1,2\}$	9	**17**	yes	$\{1,3,4\}$	11	20	no
$\{1,3\}$	8	16	yes	$\{2,3,4\}$	10	17	no
$\{1,4\}$	8	14	yes	$\{1,2,3,4\}$	15	27	no

Among the 11 feasible subsets, the best one is $\tilde{S} = \{1,2\}$, with $z(I) = 17$. It is the only optimal solution: the infeasible subset $\{2,3,4\}$ also has profit 17, but weight $10 > 9$.

1.4.1 Exponential growth

- Brute force works for the small instances above, but the number of candidates grows very quickly with the size n of the instance. The following table compares polynomial and exponential growth.

n	n^2	n^3	2^n	$(n-1)!/2$	$B(n)$
5	25	125	32	12	52
10	100	1 000	1 024	181 440	115 975
15	225	3 375	32 768	$\approx 4.36 \cdot 10^{10}$	$\approx 1.38 \cdot 10^{9}$
20	400	8 000	1 048 576	$\approx 6.08 \cdot 10^{16}$	$\approx 5.17 \cdot 10^{13}$
25	625	15 625	$\approx 3.36 \cdot 10^{7}$	$\approx 3.10 \cdot 10^{23}$	$\approx 4.64 \cdot 10^{18}$
30	900	27 000	$\approx 1.07 \cdot 10^{9}$	$\approx 4.42 \cdot 10^{30}$	$\approx 8.47 \cdot 10^{23}$



1.4.2 Computing times

- We assume that a computer evaluates 10^9 candidate solutions per second, and we compute the time of a complete enumeration (one year = 365.25 days):

n	2^n	time at 10^9 per second	time at 10^{12} per second
10	1 024	$\approx$ 1.02 microseconds	$\approx$ 1.02 nanoseconds
20	$\approx 1.05 \cdot 10^6$	$\approx$ 1.05 milliseconds	$\approx$ 1.05 microseconds
30	$\approx 1.07 \cdot 10^9$	$\approx$ 1.07 seconds	$\approx$ 1.07 milliseconds
40	$\approx 1.10 \cdot 10^{12}$	$\approx$ 18.3 minutes	$\approx$ 1.10 seconds
50	$\approx 1.13 \cdot 10^{15}$	$\approx$ 13.0 days	$\approx$ 18.8 minutes
60	$\approx 1.15 \cdot 10^{18}$	$\approx$ 36.5 years	$\approx$ 13.3 days
70	$\approx 1.18 \cdot 10^{21}$	$\approx 3.74 \cdot 10^4$ years	$\approx$ 37.4 years
80	$\approx 1.21 \cdot 10^{24}$	$\approx 3.83 \cdot 10^7$ years	$\approx 3.83 \cdot 10^4$ years

	TSP: $(n-1)!/2$ tours			partitions: $B(n)$	
n	number	10^9 per second	10^{12} per second	number	10^9 per second
10	181 440	$\approx$ 181 microseconds	$\approx$ 181 nanoseconds	115 975	$\approx$ 116 microseconds
15	$\approx 4.36 \cdot 10^{10}$	$\approx$ 43.6 seconds	$\approx$ 43.6 milliseconds	$\approx 1.38 \cdot 10^9$	$\approx$ 1.38 seconds
20	$\approx 6.08 \cdot 10^{16}$	$\approx$ 1.93 years	$\approx$ 16.9 hours	$\approx 5.17 \cdot 10^{13}$	$\approx$ 14.4 hours
25	$\approx 3.10 \cdot 10^{23}$	$\approx 9.83 \cdot 10^6$ years	$\approx 9.83 \cdot 10^3$ years	$\approx 4.64 \cdot 10^{18}$	$\approx$ 147 years

- A computer 1000 times faster does not change the picture: since $2^{10} = 1024 > 1000$, it enumerates in the same time the subsets of only about 10 more items. For the TSP the gain is even smaller: going from n to $n+1$ cities multiplies the number of tours by n, and $15 \cdot 16 \cdot 17 = 4080 > 1000$, so from 15 cities the faster computer gains fewer than 3 cities.

Complete enumeration is possible only for **very small instances**. Faster computers do not help: an exponential number of candidates defeats any constant speed-up. We need smarter approaches, which **exclude** whole parts of the solution space without exploring them.

1.5 Primal bounds, dual bounds and gap

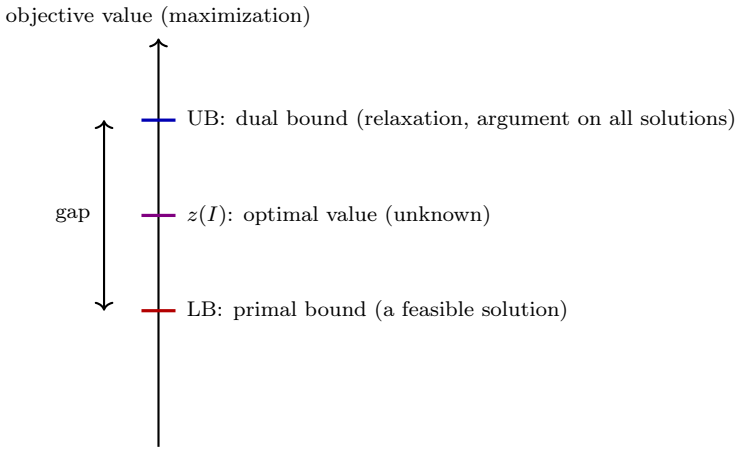
- The smarter approaches rest on a simple idea: instead of computing $z(I)$ directly, we **enclose** it between two numbers. One comes from a feasible solution, the other from an argument that holds for *all* feasible solutions.

Definition 1.10: primal and dual bounds

Consider an instance I of a combinatorial optimization problem with feasible solutions $\mathcal{F}$, objective function f and optimal value $z(I)$.

1. A **primal bound** is the value $f(\bar{S})$ of a feasible solution $\bar{S} \in \mathcal{F}$.
2. A **dual bound** is a number β such that $f(S) \leq \beta$ for all $S \in \mathcal{F}$ (maximization form), or $f(S) \geq \beta$ for all $S \in \mathcal{F}$ (minimization form).

- In the **maximization form** a primal bound is a **lower bound** and a dual bound is an **upper bound**:
$$\text{LB} = f(\bar{S}) \;\leq\; z(I) \;\leq\; \text{UB}.$$
- In the **minimization form** the roles are exchanged: a primal bound is an **upper bound** $\text{UB} = f(\bar{S})$ and a dual bound is a **lower bound** LB, with $\text{LB} \leq z(I) \leq \text{UB}$.
- Primal bounds come from feasible solutions, for example found by **heuristic algorithms**; dual bounds come from **relaxations** (for example the linear programming relaxation) or from combinatorial arguments.



Definition 1.11: gap

Let $\text{LB} \le z(I) \le \text{UB}$ be a lower and an upper bound of an instance I, one primal and one dual. The **absolute gap** is $\text{UB} - \text{LB}$ and, for $\text{LB} \neq 0$, the **relative gap** (or simply **gap**) is

$$gap(I) = \frac{\text{UB} - \text{LB}}{|\text{LB}|}.$$

- The denominator is always the **lower bound**: in the maximization form it is the value of the feasible solution, as in the optimality gap of the integer linear programming module; in the minimization form it is the dual bound.
- With this choice the gap is never smaller than the relative error of the feasible solution with respect to $z(I)$ (chapter «Heuristic algorithms»): a gap of 5% guarantees a solution within 5% of the optimum.
- If the gap is zero, the feasible solution $\bar{S}$ is optimal and the dual bound is a **certificate** of its optimality.

Observation 1.12

If $f(S) \in \mathbb{Z}$ for every $S \in \mathcal{F}$, then in the maximization form $\lfloor \beta \rfloor$ is also a dual bound, and in the minimization form $\lceil \beta \rceil$ is also a dual bound.

Proof. In the maximization form, for every $S \in \mathcal{F}$ we have $f(S) \le \beta$ and $f(S) \in \mathbb{Z}$; the largest integer not greater than β is $\lfloor \beta \rfloor$, hence $f(S) \le \lfloor \beta \rfloor$. The minimization form is analogous. □

Example 1.9: bounds on the small instances

Knapsack (Example 1.1, maximization).

- *Primal bound.* The greedy algorithm inserts the items by non-increasing efficiency p_j/w_j: the efficiencies are 2, $\frac{7}{4}$, 2, $\frac{4}{3}$, so the order is $1, 3, 2, 4$ (ties by index). It inserts items 1 and 3 (residual capacity 1), and items 2 and 4 do not fit: $\text{LB} = 16$.
- *Dual bound.* The sum of all profits gives 27. A better bound is the continuous relaxation, where items can be taken fractionally: items 1 and 3 fill 8 units and $\frac{1}{4}$ of item 2 fills the last one,

$$\text{UB} = 10 + 6 + \tfrac{1}{4} \cdot 7 = \tfrac{71}{4} = 17.75.$$

- The profits are integer, so $\lfloor 17.75 \rfloor = 17$ is a dual bound (Observation 1.12). The gap between 16 and 17 is 1 (relative gap $\frac{1}{16} = 6.25\%$). The solution $\{1, 2\}$ of value 17 closes it, and it is optimal.

Bin packing (Example 1.2, minimization).

- *Primal bound.* First Fit puts each item into the first bin where it fits and returns $\{1, 4\}, \{2, 5\}, \{3\}, \{6\}$: $\text{UB} = 4$.

- *Dual bound.* Each bin holds at most 7 of the total weight 21, so LB $= \lceil 21/7 \rceil = 3$. The solution of Example 1.2 uses 3 bins and is optimal.

Scheduling (Example 1.5, minimization).

- *Primal bound.* The LPT rule (longest job first, on the least loaded machine) gives the loads $11, 8, 8$: UB $= 11$.
- *Dual bound.* The makespan is at least the average load and at least the longest job: LB $= \max\{\lceil 27/3 \rceil, 5\} = 9$. The schedule of Example 1.5 has makespan 9 and is optimal.

TSP (Example 1.4, minimization).

- *Primal bound.* The nearest neighbour heuristic from city 1 builds the tour $(1, 3, 2, 4, 5)$ of length 25: UB $= 25$.
- *Dual bound.* Each city is entered and left by two edges of the tour, at least as long as its two shortest edges; summing over the cities counts every edge twice:

$$\text{LB} = \left\lceil \tfrac{1}{2}\big((2+2)+(3+4)+(2+4)+(2+3)+(4+7)\big) \right\rceil = \left\lceil \tfrac{33}{2} \right\rceil = 17.$$

- The optimal tour has length 18: a gap of 1 remains, and only the enumeration of Example 1.6 certifies the optimum.

- These bounds start the **dashboard** of the course: for each instance we record the best primal and dual bounds found so far. Every following chapter improves one side or the other.

instance	sense	primal bound (heuristic)	dual bound (argument)	optimum
KP	max	16 (greedy)	17 ($\lfloor 71/4 \rfloor$)	17
BPP	min	4 (First Fit)	3 ($\lceil 21/7 \rceil$)	3
$P\|\|C_{\max}$	min	11 (LPT)	9 (average load)	9
TSP	min	25 (nearest neighbour)	17 (two shortest edges)	18

1.6 A glimpse of computational complexity

- Brute force takes exponential time. Is there an algorithm that solves these problems in polynomial time? Computational complexity classifies problems according to this question; here we give only the definitions needed in the course.
- The **size** of an instance is the number of bits needed to write it. For example, a knapsack instance has size of the order of $n \log_2(\max_j\{p_j, w_j\}) + \log_2 c$: a number c is written with about $\log_2 c$ bits, not with c symbols.
- An algorithm is **polynomial** if its running time is bounded by a polynomial in the size of the instance.
- The **decision version** of an optimization problem asks a yes/no question, for example: "is there a feasible subset with profit at least k?".

Definition 1.13: the classes P and NP

1. **P** is the class of decision problems that can be solved by a polynomial algorithm.
2. **NP** is the class of decision problems whose "yes" instances admit a certificate that can be checked in polynomial time (for the knapsack: a subset, whose weight and profit are computed in time linear in n).
3. A decision problem in NP is **NP-complete** if every problem in NP can be reduced to it in polynomial time. An optimization problem is **NP-hard** if its decision version is NP-complete (more generally, if a polynomial algorithm for it would give a polynomial algorithm for an NP-complete problem).

- It is not known whether $P = NP$. No polynomial algorithm is known for any NP-complete problem, and it is widely believed that none exists: for an NP-hard problem we do not expect a polynomial exact algorithm.

1.6.1 Pseudo-polynomial algorithms and strong NP-hardness

- Some NP-hard problems can be solved in time polynomial in n **and in the value** of the numbers of the instance. For example, the knapsack problem with integer data can be solved by dynamic programming in $O(n\,c)$ time (Part III of the course). This is not polynomial: c is exponential in its number of bits $\log_2 c$.

Definition 1.14: pseudo-polynomial algorithm, weak and strong NP-hardness

1. An algorithm is **pseudo-polynomial** if its running time is bounded by a polynomial in the size of the instance and in the largest number appearing in it.
2. An NP-hard problem is **strongly NP-hard** if it remains NP-hard when all its numbers are bounded by a polynomial in the size of the instance (equivalently, when the numbers are written in unary). Otherwise, if it admits a pseudo-polynomial algorithm, it is called **weakly NP-hard** (NP-hard in the ordinary sense).

- A strongly NP-hard problem has no pseudo-polynomial algorithm, unless $P = NP$.
- The six problems of the course are all NP-hard, but not all in the same way:

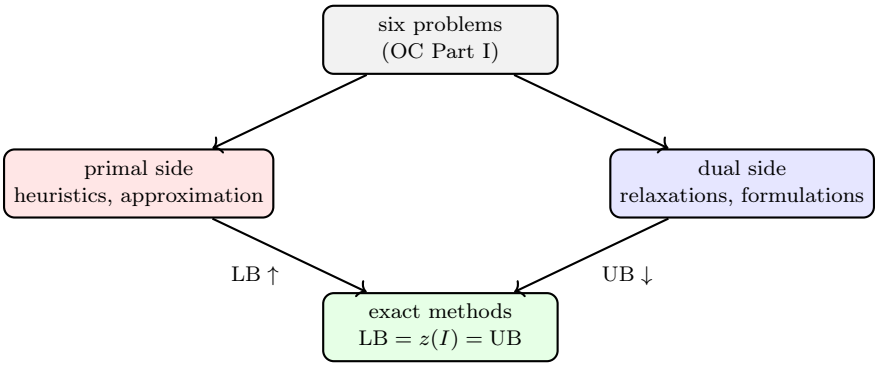
Problem	**Complexity**	**Remark**
knapsack	weakly NP-hard	pseudo-polynomial dynamic programming in $O(n\,c)$
bin packing	strongly NP-hard	no pseudo-polynomial algorithm unless $P = NP$
vertex coloring	strongly NP-hard	deciding whether $\chi(G) \leq 3$ is NP-complete; the instance contains no numbers
TSP	strongly NP-hard	contains the Hamiltonian cycle problem
$P\|\|C_{\max}$	strongly NP-hard (m part of the input)	weakly NP-hard for a fixed number of machines, e.g. $m = 2$
lot sizing	NP-hard (capacitated)	the uncapacitated single-item case is polynomial (Wagner–Whitin); the capacitated single-item case with integer data has a pseudo-polynomial dynamic program

NP-hardness does not mean that an instance cannot be solved: it means that no algorithm is expected to solve *every* instance in polynomial time. In practice we combine **exact methods** that close the gap between primal and dual bounds, **heuristics** that find good solutions quickly, and **approximation algorithms** that guarantee a bounded gap in the worst case.

1.7 Roadmap of the course

- The guiding question of the course is: **how do we solve a combinatorial optimization problem, exactly or not, and how do we certify how far we are from the optimum?**
- Each stage answers a question and works on the primal side, on the dual side, or on both, always on the same six problems.

Stage	Guiding question	Where
The problems	What do we want to solve, and why is it hard?	OC Part I (this chapter, the six problems)
The language	How do we write it?	integer linear programming; module *MIP modelling*
The primal side	How do I find a good solution quickly?	heuristics (OC Part IV), MIP heuristics
The dual side	How do I prove that no better solution exists?	linear relaxation and dual bounds
Stronger formulations	How do I make the bound tighter?	ideal formulations, valid inequalities, covers; formulations of the six problems (OC Part II)
Closing the gap	How do I find the optimum and certify it?	cutting planes, branch and bound, branch and cut, MIP solvers; dynamic programming (OC Part III)
The guarantees	How wrong can a heuristic be, in the worst case?	approximation algorithms (OC Part V)
Tailor-made	What do we gain by knowing the problem?	specialized algorithms (OC Part VI)



- The arrows refer to a maximization problem: the primal side raises the lower bound, the dual side lowers the upper bound, and an exact method stops when the two meet.

1.8 Typical mistakes

Warning

- **Confusing candidate and feasible solutions.** The numbers 2^n, $(n-1)!/2$ and $B(n)$ count the candidates; the feasible solutions can be far fewer, for example 11 subsets out of 16 in Example 1.1.
- **Exchanging the two directions of the bounds.** In a maximization problem a primal bound is a lower bound and a dual bound is an upper bound; in a minimization problem the roles are exchanged.
- **Rounding a dual bound when the objective is not integer.** The rounding of Observation 1.12 needs $f(S) \in \mathbb{Z}$ for every feasible solution.
- **Counting equivalent solutions twice.** A tour and its reverse are the same tour, and two assignments that differ by a renaming of identical machines are the same schedule.
- **Reading NP-hard as unsolvable.** NP-hardness is a statement on the worst case: many instances of practical size are solved, and the rest of the course shows how.

1.9 In summary

Summary

- A combinatorial optimization problem has a **finite** set of feasible solutions, which are subsets, orderings, partitions or assignments.
- The six problems of the course are knapsack (a subset), bin packing and vertex coloring (a partition), TSP (a cyclic ordering), $P\|C_{\max}$ (an assignment) and capacitated lot sizing (a subset of setup periods).
- The number of candidate solutions is 2^n for subsets, $(n-1)!/2$ for tours and $B(n)$ for partitions: brute force is limited to very small instances, even on much faster computers.
- A **primal bound** is the value of a feasible solution, a **dual bound** holds for all feasible solutions; the optimum lies between them, and a zero **gap** certifies optimality.
- All six problems are NP-hard; the knapsack problem is weakly NP-hard (pseudo-polynomial dynamic programming), bin packing is strongly NP-hard.

Dashboard after this chapter.

instance	lower bound	upper bound	from
KP, Example 1.1 (max)	16 (greedy)	17 ($\lfloor 71/4 \rfloor$)	this chapter
BPP, Example 1.2 (min)	3 ($\lceil 21/7 \rceil$)	4 (First Fit)	this chapter
$P\|\|C_{\max}$, Example 1.5 (min)	9 (average load)	11 (LPT)	this chapter
TSP, Example 1.4 (min)	17 (two shortest edges)	25 (nearest neighbour)	this chapter

1.10 Exercises

Exercise 1.1 — Recognizing the problems (basic)

For each situation, name the combinatorial object of a solution (subset, ordering, partition, assignment), the problem of the course it corresponds to, and the number of candidate solutions.

Questions

1. A research fund must choose which of 12 proposals to finance with a fixed budget.
2. A technician must visit 8 sites (her office included) and come back to the office.
3. 8 boxes must be loaded into identical vans of the same capacity, using as few vans as possible (count the ways of grouping the boxes).
4. 8 jobs must be assigned to 3 identical machines (count the assignments).
5. A factory must decide in which of the next 12 weeks to set up its line.

Solution

1. A subset of proposals, knapsack problem: $2^{12} = 4096$ subsets.
2. A cyclic ordering of the 8 sites, TSP: $(8-1)!/2 = 5040/2 = 2520$ tours.
3. A partition of the boxes, bin packing problem: $B(8) = 4140$ partitions.
4. An assignment of jobs to machines, $P\|\|C_{\max}$: $3^8 = 6561$ assignments.
5. A subset of weeks, lot sizing: $2^{12} = 4096$ choices of the setup periods.

Exercise 1.2 — Brute force, greedy and bound on a knapsack (standard)

Consider the knapsack instance with $n = 4$ items, capacity $c = 10$, profits $\boldsymbol{p} = (7, 6, 5, 4)$ and weights $\boldsymbol{w} = (6, 5, 4, 3)$.

Questions

1. Solve the instance by brute force, listing the 16 subsets with weight and profit.
2. Apply the greedy algorithm by non-increasing efficiency p_j/w_j and compute the rel-

ative gap between its value and the optimum.

3. Compute the dual bound of the continuous relaxation and round it.

Solution

1. The subsets are:

S	weight	profit	feasible	S	weight	profit	feasible
$\emptyset$	0	0	yes	$\{2,3\}$	9	11	yes
$\{1\}$	6	7	yes	$\{2,4\}$	8	10	yes
$\{2\}$	5	6	yes	$\{3,4\}$	7	9	yes
$\{3\}$	4	5	yes	$\{1,2,3\}$	15	18	no
$\{4\}$	3	4	yes	$\{1,2,4\}$	14	17	no
$\{1,2\}$	11	13	no	$\{1,3,4\}$	13	16	no
$\{1,3\}$	10	**12**	yes	$\{2,3,4\}$	12	15	no
$\{1,4\}$	9	11	yes	$\{1,2,3,4\}$	18	22	no

   There are 10 feasible subsets; the optimal solution is $\tilde{S} = \{1,3\}$ with $z(I) = 12$.

2. The efficiencies are $\frac{7}{6}, \frac{6}{5}, \frac{5}{4}, \frac{4}{3}$, so the order is $4,3,2,1$. The greedy algorithm inserts item 4 (residual capacity 7) and item 3 (residual capacity 3); items 2 and 1 do not fit. It returns $\{3,4\}$ with value 9. The relative gap with respect to the optimum is $\frac{12-9}{9} = \frac{1}{3}$.

3. The continuous relaxation takes items 4 and 3 and $\frac{3}{5}$ of item 2: $\text{UB} = 4+5+\frac{3}{5}\cdot 6 = \frac{63}{5} = 12.6$. Since profits are integer, $\lfloor 12.6 \rfloor = 12$ is a dual bound, equal to the optimum: the solution $\{1,3\}$ is certified optimal without enumeration.

Exercise 1.3 — The tours of a small TSP (basic)

Consider 4 cities with symmetric distances $d_{12} = 4$, $d_{13} = 6$, $d_{14} = 5$, $d_{23} = 3$, $d_{24} = 7$, $d_{34} = 2$.

Questions

1. List all distinct tours with their length and find an optimal tour.

2. How many distinct tours are there with 6 cities? And with 10 cities?

Solution

1. There are $(4-1)!/2 = 3$ tours, all starting from city 1:

$$\begin{aligned} (1,2,3,4): &\quad 4+3+2+5 = 14, \\ (1,2,4,3): &\quad 4+7+2+6 = 19, \\ (1,3,2,4): &\quad 6+3+7+5 = 21. \end{aligned}$$

   The optimal tour is $(1,2,3,4)$, with length 14.

2. With 6 cities: $5!/2 = 60$ tours; with 10 cities: $9!/2 = 181\,440$ tours.

Exercise 1.4 — How large can n be? (standard)

A computer evaluates 10^9 candidate solutions per second, and we allow one hour of computation.

Questions

1. What is the largest number n of items for which all subsets of a knapsack instance can be enumerated?
2. What is the largest number n of cities for which all tours of a TSP instance can be enumerated?
3. Answer the same questions for a computer 1000 times faster.

Solution

In one hour the computer evaluates $3600 \cdot 10^9 = 3.6 \cdot 10^{12}$ candidates.

1. $2^{41} \approx 2.20 \cdot 10^{12} \leq 3.6 \cdot 10^{12} < 2^{42} \approx 4.40 \cdot 10^{12}$, hence $n = 41$ items.
2. For 16 cities $15!/2 \approx 6.54 \cdot 10^{11} \leq 3.6 \cdot 10^{12}$, while for 17 cities $16!/2 \approx 1.05 \cdot 10^{13} > 3.6 \cdot 10^{12}$; hence $n = 16$ cities.
3. The budget becomes $3.6 \cdot 10^{15}$. For the knapsack, $2^{51} \approx 2.25 \cdot 10^{15} \leq 3.6 \cdot 10^{15} < 2^{52} \approx 4.50 \cdot 10^{15}$, hence $n = 51$ items: 10 more. For the TSP, $18!/2 \approx 3.20 \cdot 10^{15} \leq 3.6 \cdot 10^{15} < 19!/2 \approx 6.08 \cdot 10^{16}$, hence $n = 19$ cities: only 3 more.

Exercise 1.5 — LPT, bounds and gap on $P\|C_{\max}$ (standard)

Consider the instance of $P\|C_{\max}$ with $m = 2$ machines and $n = 5$ jobs with processing times $\boldsymbol{t} = (5, 4, 3, 3, 3)$.

Questions

1. Apply the LPT rule (jobs by non-increasing processing time, each one on the least loaded machine, ties broken by the smallest machine index) and give the makespan: is it a primal or a dual bound?
2. Compute the dual bound $\max\{\lceil \sum_j t_j/m \rceil, \max_j t_j\}$ and the absolute and relative gaps.
3. Find an optimal schedule and explain why it is optimal.

Solution

1. LPT assigns job 1 (5) to M_1 and job 2 (4) to M_2; job 3 (3) to M_2 (load 7); job 4 (3) to M_1 (load 8); job 5 (3) to M_2 (load 10). Hence $J_1 = \{1, 4\}$, $J_2 = \{2, 3, 5\}$ and $C_{\max} = 10$. It is the value of a feasible solution of a minimization problem, hence a primal bound: UB $= 10$.

2. $\sum_j t_j = 18$, hence LB $= \max\{\lceil 18/2 \rceil, 5\} = 9$. The absolute gap is $10 - 9 = 1$ and the relative gap is $\frac{1}{9} \approx 11.1\%$.

3. The schedule $J_1 = \{1, 2\}$, $J_2 = \{3, 4, 5\}$ has loads $5 + 4 = 9$ and $3 + 3 + 3 = 9$, hence $C_{\max} = 9 =$ LB. The gap is zero, so this schedule is optimal and $z(I) = 9$.

Chapter 2

Knapsack problems

Prerequisites

- From this module, the chapter «Combinatorial optimization problems»: instances, feasible solutions, primal and dual bounds and the gap.
- From the *Preliminaries*: subsets of a finite set and the number 2^n of subsets.

Objectives

At the end of this chapter you will be able to:

- state the definition of the knapsack problem and of its variants, with instances and feasible solutions;
- explain the assumptions on the data and why they can be made without loss of generality;
- count the candidate solutions and the feasible solutions of a small instance;
- find a feasible solution by hand and compute a simple combinatorial dual bound;
- tell the variants apart: copies of the items, several resources, several knapsacks, classes of items.

2.1 The knapsack problem

Definition 2.1: Knapsack Problem

Given:

1. a knapsack of *capacity* $c \in \mathbb{Z}_{\geq 1}$,
2. a set $\mathscr{I} = \{1, 2, \ldots, n\}$ of n items,
3. for each item $j \in \mathscr{I}$, a *profit* $p_j \in \mathbb{Z}_{\geq 1}$ and a *weight* $w_j \in \mathbb{Z}_{\geq 1}$,

the **Knapsack Problem** (KP) asks to decide which items to insert into the knapsack, so that the total weight of the selected items does not exceed the knapsack capacity, and the total profit of the selected items is maximized.

- A **KP instance** is specified by:
  1. two integer numbers:
     the number of items $n \in \mathbb{Z}_{\geq 1}$,
     the knapsack capacity $c \in \mathbb{Z}_{\geq 1}$,
  2. two vectors of integer numbers of length n:
     $\boldsymbol{p} \in \mathbb{Z}^n_{\geq 1}$, containing the item profits,
     $\boldsymbol{w} \in \mathbb{Z}^n_{\geq 1}$, containing the item weights.
- A **KP feasible solution** is a subset of items

$$S \subseteq \mathscr{I}$$

such that the total weight of the selected items does not exceed the knapsack capacity, namely

$$\sum_{j \in S} w_j \leq c.$$

Accordingly, the KP can be equivalently stated as the problem of selecting a subset of items whose total weight respects the knapsack capacity, and whose total profit is maximized.

Given a KP instance, the KP can be formally stated as:

$$\tilde{S} \in \arg\max_{S \subseteq \mathscr{I}} \left\{ \sum_{j \in S} p_j \ : \ \sum_{j \in S} w_j \leq c \right\}. \tag{2.1}$$

Example 2.1: KP instance

We consider a knapsack of capacity $c = 9$ and a set $\mathscr{I} = \{1, 2, 3, 4, 5, 6\}$ of 6 items ($n = 6$), with the following profits and weights:

Items		$j=1$	$j=2$	$j=3$	$j=4$	$j=5$	$j=6$
Profits	p_j	6	5	8	9	6	7
Weights	w_j	2	3	6	7	5	9

We call this KP instance Example 2.1: it is the reference instance of the chapter.

2.1.1 Assumptions

- We make the following assumptions:

1. The total weight of all items exceeds the knapsack capacity (otherwise, all items could be selected). Formally:
$$\sum_{j \in \mathscr{I}} w_j > c$$
2. Each item individually fits into the knapsack (otherwise, it cannot be selected). Formally:
$$w_j \leq c, \quad \forall j \in \mathscr{I}$$

2.1.2 Counting the solutions, a first solution and a simple bound

- **Candidate solutions.** Every subset of $\mathscr{I}$ is a candidate solution, so there are 2^n of them (Chapter «Combinatorial optimization problems»). For Example 2.1 we have $2^6 = 64$.
- **Feasible solutions.** The three lightest items weigh $2+3+5 = 10 > 9$, so no subset with three or more items is feasible. The feasible solutions are the empty set, the 6 singletons and the 6 pairs of the following table: 13 solutions out of 64.

pair S	$\{1,2\}$	$\{1,3\}$	$\{1,4\}$	$\{1,5\}$	$\{2,3\}$	$\{2,5\}$
weight	5	8	9	7	9	8
profit	11	14	**15**	12	13	11

- **A first solution, by eye.** Item 4 has the largest profit (9, weight 7). The residual capacity $9 - 7 = 2$ is enough for item 1 (weight 2, profit 6). The subset $\{1, 4\}$ has weight 9 and profit 15: it is a **primal bound**, $\text{LB} = 15 \leq z(I)$.

Observation 2.2: cardinality bound

Let $\bar{k}$ be the largest integer such that the $\bar{k}$ lightest items weigh at most c. Every feasible subset has at most $\bar{k}$ items, hence $z(I)$ is at most the sum of the $\bar{k}$ largest profits.

Proof. A subset with k items weighs at least as much as the k lightest items. If $k > \bar{k}$, the k lightest items weigh more than c, so the subset is not feasible. The profit of a subset with at most $\bar{k}$ items is at most the sum of the $\bar{k}$ largest profits. □

- **A first dual bound.** In Example 2.1 we have $\bar{k} = 2$, since $2+3 = 5 \leq 9$ and $2+3+5 = 10 > 9$. The two largest profits are 9 and 8, so UB = 17. The sum of all the profits, 41, is a much weaker dual bound.
- **The gap.** We have $15 \leq z(I) \leq 17$: the absolute gap is 2 and the relative gap is $\frac{2}{15}$. Listing the 13 feasible subsets shows that $\{1, 4\}$ is the unique optimal solution and $z(I) = 15$; for larger instances we need stronger bounds, which come from relaxations (module *Integer linear programming* and Chapter «Knapsack: formulations and bounds»).

2.2 The subset sum problem

Definition 2.3: Subset Sum Problem

Given:

1. a target value $c \in \mathbb{Z}_{\geq 1}$,
2. a set $\mathscr{I} = \{1, 2, \ldots, n\}$ of n items,
3. for each item $j \in \mathscr{I}$, a *weight* $w_j \in \mathbb{Z}_{\geq 1}$,

the **Subset Sum Problem** (SSP) asks whether there exists a subset of items whose total weight is exactly equal to the target value c.

- An **SSP instance** is specified by:
  1. two integer numbers:
     the number of items $n \in \mathbb{Z}_{\geq 1}$,
     the target value $c \in \mathbb{Z}_{\geq 1}$,
  2. one vector of integer numbers of length n:
     $\boldsymbol{w} \in \mathbb{Z}^n_{\geq 1}$, containing the item weights.
- An **SSP feasible solution** is a subset of items

$$S \subseteq \mathscr{I}$$

such that the total weight of the selected items is exactly equal to the target value, namely

$$\sum_{j \in S} w_j = c.$$

Accordingly, the SSP can be equivalently stated as the problem of selecting a subset of items whose total weight matches the target value exactly.

Given an SSP instance, the SSP can be formally stated as:

$$\text{find } S \subseteq \mathscr{I} \quad \text{such that} \quad \sum_{j \in S} w_j = c. \tag{2.2}$$

Example 2.2: SSP instance

We consider a target value $c = 9$ and a set $\mathscr{I} = \{1, 2, 3, 4, 5, 6\}$ of 6 items ($n = 6$), with the following weights:

Items		$j=1$	$j=2$	$j=3$	$j=4$	$j=5$	$j=6$
Weights	w_j	2	3	6	7	5	9

For instance, the subset $S = \{1, 4\}$ is feasible since $w_1 + w_4 = 2 + 7 = 9$. Also, the subset $S = \{6\}$ is feasible since $w_6 = 9$.

- The SSP is not an **optimization problem**, since its answer is either **yes** or **no**. However, it can be solved by using an optimization problem.
- An SSP instance can be solved as a KP instance by setting the profit of each item equal to its weight, namely

  $$p_j = w_j, \qquad \forall j \in \mathscr{I}.$$

  In this case, we solve the following KP:

  $$\tilde{S} \in \arg\max_{S \subseteq \mathscr{I}} \left\{ \sum_{j \in S} w_j \;\; : \;\; \sum_{j \in S} w_j \leq c \right\}.$$

  If $\sum_{j \in \tilde{S}} w_j = c$, then the answer to the SSP is **yes**; otherwise, if $\sum_{j \in \tilde{S}} w_j < c$, the answer is **no**.

2.2.1 Assumptions

- We make the following assumptions:

  1. The total weight of all items is at least the target value (otherwise, no feasible subset can exist). Formally:

     $$\sum_{j \in \mathscr{I}} w_j \geq c$$

  2. Each item weight is not larger than the target value (otherwise, it cannot belong to any feasible subset). Formally:

     $$w_j \leq c, \qquad \forall j \in \mathscr{I}$$

2.3 The bounded knapsack problem

Definition 2.4: Bounded Knapsack Problem

Given:

1. a knapsack of *capacity* $c \in \mathbb{Z}_{\geq 1}$,
2. a set $\mathscr{I} = \{1, 2, \ldots, n\}$ of n items,
3. for each item $j \in \mathscr{I}$, a *profit* $p_j \in \mathbb{Z}_{\geq 1}$, a *weight* $w_j \in \mathbb{Z}_{\geq 1}$, and a *number of available copies* $b_j \in \mathbb{Z}_{\geq 1}$,

the **Bounded Knapsack Problem** (BKP) asks to decide how many copies of each item to insert into the knapsack, so that the total weight of all selected item copies does not exceed the knapsack capacity, each item is taken no more than its available copies, and the total profit of all selected item copies is maximized.

- A **BKP instance** is specified by:
  1. two integer numbers:
     the number of items $n \in \mathbb{Z}_{\geq 1}$,
     the knapsack capacity $c \in \mathbb{Z}_{\geq 1}$,
  2. three vectors of integer numbers of length n:
     $\boldsymbol{p} \in \mathbb{Z}^n_{\geq 1}$, containing the item profits,
     $\boldsymbol{w} \in \mathbb{Z}^n_{\geq 1}$, containing the item weights,
     $\boldsymbol{b} \in \mathbb{Z}^n_{\geq 1}$, containing the number of available copies of each item.
- Given a BKP instance, we define the **extended set of items** as

$$\hat{\mathscr{I}} = \big\{ (j,k) : j \in \mathscr{I},\ k \in \{1, 2, \ldots, b_j\} \big\}, \tag{2.3}$$

  where each pair (j,k) represents the k-th available copy of item j. The cardinality of the extended set of items is

$$|\hat{\mathscr{I}}| = \sum_{j \in \mathscr{I}} b_j.$$

- A **BKP feasible solution** is a subset of items

$$S \subseteq \hat{\mathscr{I}}$$

  such that the total weight of the selected items does not exceed the knapsack capacity, namely

$$\sum_{(j,k) \in S} w_j \leq c.$$

  Accordingly, the BKP can be equivalently stated as the problem of selecting a subset of the extended set of items whose total weight respects the knapsack capacity, and whose total profit is maximized.

Given a BKP instance, the BKP can be formally stated as:

$$\tilde{S} \in \arg\max_{S \subseteq \hat{\mathscr{I}}} \left\{ \sum_{(j,k)\in S} p_j \ : \ \sum_{(j,k)\in S} w_j \leq c \right\}. \tag{2.4}$$

Example 2.3: BKP instance

We consider a knapsack of capacity $c = 9$ and a set $\mathscr{I} = \{1, 2, 3, 4, 5, 6\}$ of 6 items ($n = 6$), with the following profits, weights, and available copies:

Items		$j=1$	$j=2$	$j=3$	$j=4$	$j=5$	$j=6$
Profits	p_j	6	5	8	9	6	7
Weights	w_j	2	3	6	7	5	9
Available copies	b_j	2	2	1	1	1	1

The extended set of items is:

$$\hat{\mathscr{I}} \ = \ \{ \ (1,1), (1,2), \ (2,1), (2,2), \ (3,1), \ (4,1), \ (5,1), \ (6,1) \ \}.$$

Each copy (j, k) has profit p_j and weight w_j.

Copy label	1	2	3	4	5	6	7	8
(j,k)	$(1,1)$	$(1,2)$	$(2,1)$	$(2,2)$	$(3,1)$	$(4,1)$	$(5,1)$	$(6,1)$
Profit p_j	6	6	5	5	8	9	6	7
Weight w_j	2	2	3	3	6	7	5	9

2.3.1 Assumptions

- We make the following assumptions:

1. The total weight of all available copies of all items exceeds the knapsack capacity (otherwise, all copies could be selected). Formally:
$$\sum_{j\in\mathscr{I}} b_j \, w_j > c$$
2. Each item individually fits into the knapsack (otherwise, it cannot be selected). Formally:
$$w_j \leq c, \qquad \forall j \in \mathscr{I}$$
3. The total weight of all available copies of any single item does not exceed the knapsack capacity (otherwise, that item could never be taken in the maximum allowed number of copies). Formally:
$$b_j \, w_j \leq c, \qquad \forall j \in \mathscr{I}$$

2.4 The unbounded knapsack problem

Definition 2.5: Unbounded Knapsack Problem

Given:

1. a knapsack of *capacity* $c \in \mathbb{Z}_{\geq 1}$,
2. a set $\mathscr{I} = \{1, 2, \ldots, n\}$ of n items,
3. for each item $j \in \mathscr{I}$, which is available in an unlimited number of copies, a *profit* $p_j \in \mathbb{Z}_{\geq 1}$ and a *weight* $w_j \in \mathbb{Z}_{\geq 1}$,

the **Unbounded Knapsack Problem** (UKP) asks to decide how many copies of each item to insert into the knapsack, so that the total weight of all selected item copies does not exceed the knapsack capacity and the total profit of all selected item copies is maximized.

- A **UKP instance** is specified by:
  1. two integer numbers:
     the number of items $n \in \mathbb{Z}_{\geq 1}$,
     the knapsack capacity $c \in \mathbb{Z}_{\geq 1}$,
  2. two vectors of integer numbers of length n:
     $\boldsymbol{p} \in \mathbb{Z}^n_{\geq 1}$, containing the item profits,
     $\boldsymbol{w} \in \mathbb{Z}^n_{\geq 1}$, containing the item weights.
- Given a UKP instance, we first define for each item $j \in \mathscr{I}$ the **maximum feasible number of copies**

$$\hat{b}_j = \left\lfloor \frac{c}{w_j} \right\rfloor, \tag{2.5}$$

  that is, the largest number of copies of item j that can be inserted into the knapsack without exceeding its capacity.
- We then define the **extended set of items** as

$$\hat{\mathscr{I}} = \{ (j,k) : j \in \mathscr{I},\ k \in \{1, 2, \ldots, \hat{b}_j\} \}, \tag{2.6}$$

  where each pair (j,k) represents the k-th copy of item j. The cardinality of the extended set of items is

$$|\hat{\mathscr{I}}| = \sum_{j \in \mathscr{I}} \hat{b}_j.$$

- A **UKP feasible solution** is a subset of items

$$S \subseteq \hat{\mathscr{I}}$$

such that the total weight of the selected items does not exceed the knapsack capacity, namely

$$\sum_{(j,k)\in S} w_j \leq c.$$

Accordingly, the UKP can be equivalently stated as the problem of selecting a subset of the extended set of items whose total weight respects the knapsack capacity, and whose total profit is maximized.

Given a UKP instance, UKP can be formally stated as:

$$\tilde{S} = \arg\max_{S\subseteq\hat{\mathscr{I}}} \left\{ \sum_{(j,k)\in S} p_j \ : \ \sum_{(j,k)\in S} w_j \leq c \right\}. \tag{2.7}$$

Example 2.4: UKP instance

We consider a knapsack of capacity $c = 9$ and a set $\mathscr{I} = \{1, 2, 3, 4, 5, 6\}$ of 6 items ($n = 6$), with the following profits and weights:

Items		$j=1$	$j=2$	$j=3$	$j=4$	$j=5$	$j=6$
Profits	p_j	6	5	8	9	6	7
Weights	w_j	2	3	6	7	5	9

The maximum feasible number of copies of each item are:

$$\hat{b}_1 = \underbrace{\lfloor \tfrac{9}{2} \rfloor}_{=4}, \quad \hat{b}_2 = \underbrace{\lfloor \tfrac{9}{3} \rfloor}_{=3}, \quad \hat{b}_3 = \underbrace{\lfloor \tfrac{9}{6} \rfloor}_{=1}, \quad \hat{b}_4 = \underbrace{\lfloor \tfrac{9}{7} \rfloor}_{=1}, \quad \hat{b}_5 = \underbrace{\lfloor \tfrac{9}{5} \rfloor}_{=1}, \quad \hat{b}_6 = \underbrace{\lfloor \tfrac{9}{9} \rfloor}_{=1}.$$

The extended set of items is:

$$\hat{\mathscr{I}} = \{ (1,1), (1,2), (1,3), (1,4), (2,1), (2,2), (2,3), (3,1), (4,1), (5,1), (6,1) \}.$$

Each copy (j, k) has profit p_j and weight w_j.

Copy label	1	2	3	4	5	6	7	8	9	10	11
(j,k)	$(1,1)$	$(1,2)$	$(1,3)$	$(1,4)$	$(2,1)$	$(2,2)$	$(2,3)$	$(3,1)$	$(4,1)$	$(5,1)$	$(6,1)$
Profit p_j	6	6	6	6	5	5	5	8	9	6	7
Weight w_j	2	2	2	2	3	3	3	6	7	5	9

2.4.1 Assumptions

- We make the following assumptions:

  1. The total weight of all possible copies of all items (up to the maximum number that can fit in the knapsack) exceeds the knapsack capacity (otherwise, all copies could be selected). Formally:
  $$\sum_{j \in \mathscr{I}} \hat{b}_j \, w_j > c$$
  2. Each item individually fits into the knapsack (otherwise, it cannot be selected). Formally:
  $$w_j \leq c, \qquad \forall j \in \mathscr{I}$$

2.5 The multidimensional knapsack problem

Definition 2.6: Multidimensional Knapsack Problem

Given:

1. a k-dimensional knapsack,
2. for each dimension $d \in \underbrace{\{1, 2, \ldots, k\}}_{=\mathscr{D}}$, a *capacity* $c_d \in \mathbb{Z}_{\geq 1}$,
3. a set $\mathscr{I} = \{1, 2, \ldots, n\}$ of n items,
4. for each item $j \in \mathscr{I}$, a *profit* $p_j \in \mathbb{Z}_{\geq 1}$,
5. for each dimension $d \in \mathscr{D}$ and each item $j \in \mathscr{I}$, a *weight* $w_{dj} \in \mathbb{Z}_{\geq 1}$,

the **Multidimensional Knapsack Problem** (MDKP) asks to decide which items to insert into the knapsack, so that for every dimension the total weight of the selected items does not exceed the corresponding capacity, and the total profit of the selected items is maximized.

- A **MDKP instance** is specified by:

  1. two integer numbers:
     the number of items $n \in \mathbb{Z}_{\geq 1}$,
     the number of dimensions $k \in \mathbb{Z}_{\geq 1}$,
  2. one vector of integer numbers of length n:
     $\boldsymbol{p} \in \mathbb{Z}^n_{\geq 1}$, containing the item profits,
  3. one vector of integer numbers of length k:
     $\boldsymbol{c} \in \mathbb{Z}^k_{\geq 1}$, containing the knapsack capacities for each dimension,
  4. one matrix of integer numbers with k rows and n columns:
     $\boldsymbol{W} \in \mathbb{Z}^{k \times n}_{\geq 1}$, containing the the weight of the items for each dimension.

- A **MDKP feasible solution** is a subset of items

$$S \subseteq \mathscr{I}$$

such that the capacity constraints are satisfied in every dimension, namely

$$\sum_{j \in S} w_{dj} \leq c_d \qquad \forall\, d \in \mathscr{D}.$$

Accordingly, the MDKP can be equivalently stated as the problem of selecting a subset of items whose total weight respects the knapsack capacities in every dimension, and whose total profit is maximized.

Given a MDKP instance, the MDKP can be formally stated as:

$$\tilde{S} \in \arg\max_{S \subseteq \mathscr{I}} \left\{ \sum_{j \in S} p_j \;:\; \sum_{j \in S} w_{dj} \leq c_d \;\; \forall\, d \in \mathscr{D} \right\}. \tag{2.8}$$

Example 2.5: MDKP instance

We consider a 2-dimensional knapsack ($k = 2$) with capacities $c_1 = 10$ and $c_2 = 12$, and a set $\mathscr{I} = \{1, 2, 3, 4\}$ of 4 items ($n = 4$), with the following profits and weights:

Items		$j = 1$	$j = 2$	$j = 3$	$j = 4$
Profits	p_j	6	7	5	9
Weights (dim. 1)	w_{1j}	4	5	3	7
Weights (dim. 2)	w_{2j}	6	4	5	3

2.5.1 Assumptions

- We make the following assumptions:

1. The set of all items does *not* fit in the knapsack (otherwise, selecting all items is feasible and optimal). Formally,

$$\exists\, d \in \mathscr{D} \text{ such that } \sum_{j \in \mathscr{I}} w_{dj} \;>\; c_d.$$

2. Each item individually fits in the knapsack in every dimension (otherwise, it can never be selected). Formally,

$$w_{dj} \leq c_d, \qquad \forall\, d \in \mathscr{D},\ \forall\, j \in \mathscr{I}.$$

2.6 The multiple knapsack problem

Definition 2.7: Multiple Knapsack Problem

Given:

1. $m \in \mathbb{Z}_{\geq 1}$ knapsacks,
2. for each knapsack $k \in \underbrace{\{1, 2, \ldots, m\}}_{=\mathscr{K}}$, a *capacity* $c_k \in \mathbb{Z}_{\geq 1}$,
3. a set $\mathscr{I} = \{1, 2, \ldots, n\}$ of n items,
4. for each item $j \in \mathscr{I}$, a *profit* $p_j \in \mathbb{Z}_{\geq 1}$ and a *weight* $w_j \in \mathbb{Z}_{\geq 1}$,

the **Multiple Knapsack Problem** (MKP) asks to decide which items to insert into each knapsack, so that in every knapsack the total weight of the assigned items does not exceed its capacity, and the total profit of the selected items is maximized.

- A **MKP instance** is specified by:
  1. two integer numbers:
     the number of items $n \in \mathbb{Z}_{\geq 1}$,
     the number of knapsacks $m \in \mathbb{Z}_{\geq 1}$,
  2. one vector of integer numbers of length n:
     $\boldsymbol{p} \in \mathbb{Z}^n_{\geq 1}$, containing the item profits,
  3. one vector of integer numbers of length n:
     $\boldsymbol{w} \in \mathbb{Z}^n_{\geq 1}$, containing the item weights,
  4. one vector of integer numbers of length m:
     $\boldsymbol{c} \in \mathbb{Z}^m_{\geq 1}$, containing the capacities of the knapsacks.

- A **MKP feasible solution** is a family of subsets of items

$$\mathcal{S} = \{S_k\}_{k \in \mathscr{K}}, \qquad S_k \subseteq \mathscr{I} \quad \text{for all } k \in \mathscr{K},$$

such that:

(i) each item is assigned to at most one knapsack, namely

$$S_k \cap S_{k'} = \emptyset \quad \text{for all } k, k' \in \mathscr{K} \text{ with } k \neq k';$$

(ii) the total weight of the items assigned to each knapsack respects its capacity, namely

$$\sum_{j \in S_k} w_j \leq c_k \quad \text{for all } k \in \mathscr{K}.$$

Accordingly, the MKP can be equivalently stated as the problem of selecting pairwise disjoint subsets of items, one subset for each knapsack, such that every knapsack capacity is respected and the total profit is maximized.

Given a MKP instance, the MKP can be formally stated as:

$$\tilde{\mathcal{S}} \in \underset{\mathcal{S}=\{S_k\}_{k\in\mathcal{K}}}{\arg\max} \left\{ \sum_{k\in\mathcal{K}} \sum_{j\in S_k} p_j \; : \; \begin{array}{l} S_k \subseteq \mathcal{I} \quad \forall\, k \in \mathcal{K}, \\ S_k \cap S_{k'} = \emptyset \quad \forall\, k, k' \in \mathcal{K} \text{ with } k \neq k', \\ \sum_{j\in S_k} w_j \leq c_k \quad \forall\, k \in \mathcal{K} \end{array} \right\}. \tag{2.9}$$

Example 2.6: MKP instance

We consider $m = 2$ knapsacks with capacities $c_1 = 10$ and $c_2 = 12$, and a set $\mathcal{I} = \{1, 2, 3, 4, 5\}$ of 5 items ($n = 5$), with the following profits and weights:

Items		$j=1$	$j=2$	$j=3$	$j=4$	$j=5$
Profits	p_j	6	8	5	7	9
Weights	w_j	4	6	5	3	7

2.6.1 Assumptions

- We make the following assumptions:

  1. The set of all items does *not* fit in the knapsacks altogether (otherwise, selecting all items is feasible and optimal). Formally,

  $$\sum_{j\in\mathcal{I}} w_j \; > \; \sum_{k\in\mathcal{K}} c_k.$$

  2. Each item individually fits in (at least) one knapsack (otherwise, it can never be selected). Formally,

  $$\exists\, k \in \mathcal{K} \text{ such that } w_j \leq c_k, \qquad \forall\, j \in \mathcal{I}.$$

2.7 The multiple-choice knapsack problem

Definition 2.8: Multiple-Choice Knapsack Problem

Given:

1. a knapsack of *capacity* $c \in \mathbb{Z}_{\geq 1}$,
2. a set $\mathscr{I} = \{1, 2, \ldots, n\}$ of n items,
3. the set of items is partitioned into $q \in \mathbb{Z}_{\geq 2}$ classes,
4. for each class $h \in \{1, 2, \ldots, q\}$, the subset $\mathscr{J}_h \subseteq \mathscr{I}$ contains the items of that class,
5. for each item $j \in \mathscr{I}$, a *profit* $p_j \in \mathbb{Z}_{\geq 1}$ and a *weight* $w_j \in \mathbb{Z}_{\geq 1}$,

the **Multiple-Choice Knapsack Problem** (MCKP) asks to decide which items to insert into the knapsack, so that exactly one item is selected from each class, the total weight of the selected items does not exceed the knapsack capacity, and the total profit of the selected items is maximized.

- A **MCKP instance** is specified by:
  1. two integer numbers:
     the number of items $n \in \mathbb{Z}_{\geq 1}$,
     the knapsack capacity $c \in \mathbb{Z}_{\geq 1}$,
  2. one vector of integer numbers of length n:
     $\boldsymbol{p} \in \mathbb{Z}^n_{\geq 1}$, containing the item profits,
  3. one vector of integer numbers of length n:
     $\boldsymbol{w} \in \mathbb{Z}^n_{\geq 1}$, containing the item weights,
  4. one integer number:
     the number of classes $q \in \mathbb{Z}_{\geq 2}$,
  5. one family of subsets:
     $\{\mathscr{J}_1, \mathscr{J}_2, \ldots, \mathscr{J}_q\}$, a partition of $\mathscr{I}$ into q classes, where $\mathscr{J}_h \subseteq \mathscr{I}$ is the set of items belonging to class h.

- A **MCKP feasible solution** is a subset of items

$$S \subseteq \mathscr{I}$$

such that:

(i) exactly one item is selected from each class, namely

$$|S \cap \mathscr{J}_h| = 1 \qquad \text{for all } h \in \{1, 2, \ldots, q\};$$

(ii) the total weight of the selected items does not exceed the knapsack capacity, namely

$$\sum_{j \in S} w_j \leq c.$$

Accordingly, the MCKP can be equivalently stated as the problem of selecting exactly one item from each class such that the knapsack capacity is respected and the total profit is maximized.

Given a MCKP instance, the MCKP can be formally stated as:

$$\tilde{S} \in \arg\max_{S \subseteq \mathscr{I}} \left\{ \sum_{j \in S} p_j \;:\; \sum_{j \in S} w_j \leq c, \;\; |S \cap \mathscr{J}_h| = 1 \;\; \forall\, h \in \{1, 2, \ldots, q\} \right\}. \quad (2.10)$$

Example 2.7: MCKP instance

We consider a knapsack of capacity $c = 15$ and a set $\mathscr{I} = \{1, 2, 3, 4, 5, 6\}$ of 6 items ($n = 6$). The items are partitioned into $q = 3$ classes: $\mathscr{J}_1 = \{1, 2\}$, $\mathscr{J}_2 = \{3, 4\}$, $\mathscr{J}_3 = \{5, 6\}$. The profits and weights are given as follows:

Items		$j=1$	$j=2$	$j=3$	$j=4$	$j=5$	$j=6$
Profits	p_j	6	8	5	7	4	9
Weights	w_j	4	6	5	8	3	7

2.7.1 Assumptions

- We make the following assumptions:

1. Each class contains at least one item (otherwise, selecting exactly one item from each class is impossible). Formally:

$$\mathscr{J}_h \neq \emptyset \qquad \forall\, h \in \{1, \ldots, q\}.$$

2. The instance is feasible, i.e., there exists at least one choice of exactly one item per class that fits in the knapsack. A necessary and sufficient (and easy-to-check) condition is:

$$\sum_{h=1}^{q} \min_{j \in \mathscr{J}_h} w_j \;\leq\; c.$$

2.8 The knapsack problem with setups

Definition 2.9: Knapsack Problem with Setups

Given:

1. a knapsack of *capacity* $c \in \mathbb{Z}_{\geq 1}$,
2. a set $\mathscr{I} = \{1, 2, \ldots, n\}$ of n items,
3. the set of items is partitioned into $q \in \mathbb{Z}_{\geq 2}$ classes,
4. for each class $h \in \{1, 2, \ldots, q\}$, the subset $\mathscr{J}_h \subseteq \mathscr{I}$ contains the items of that class, with an associated *setup cost* $f_h \in \mathbb{Z}_{\geq 1}$ and a *setup weight* $s_h \in \mathbb{Z}_{\geq 1}$,
5. for each item $j \in \mathscr{I}$, a *revenue* $p_j \in \mathbb{Z}_{\geq 1}$ and a *weight* $w_j \in \mathbb{Z}_{\geq 1}$,

the **Knapsack Problem with Setups** (KPS) asks to decide which items to insert into the knapsack, so that the total weight of the selected items plus the setup weights of their classes does not exceed the knapsack capacity, and the total revenue of the selected items minus the setup costs of their classes is maximized.

- A **KPS instance** is specified by:
  1. two integer numbers:
     the number of items $n \in \mathbb{Z}_{\geq 1}$,
     the knapsack capacity $c \in \mathbb{Z}_{\geq 1}$,
  2. one vector of integer numbers of length n:
     $\boldsymbol{p} \in \mathbb{Z}^n_{\geq 1}$, containing the item revenues,
  3. one vector of integer numbers of length n:
     $\boldsymbol{w} \in \mathbb{Z}^n_{\geq 1}$, containing the item weights,
  4. one integer number:
     the number of classes $q \in \mathbb{Z}_{\geq 2}$,
  5. one family of subsets:
     $\{\mathscr{J}_1, \mathscr{J}_2, \ldots, \mathscr{J}_q\}$, a partition of $\mathscr{I}$ into q classes, where $\mathscr{J}_h \subseteq \mathscr{I}$ is the set of items belonging to class h,
  6. two vectors of integer numbers of length q:
     $\boldsymbol{f} \in \mathbb{Z}^q_{\geq 1}$, containing the setup costs of the classes,
     $\boldsymbol{s} \in \mathbb{Z}^q_{\geq 1}$, containing the setup weights of the classes.

- A **KPS feasible solution** is a subset of items

$$S \subseteq \mathscr{I}.$$

Given S, define the set of **activated classes** as

$$\mathscr{H}(S) \;:=\; \{\, h \in \{1,2,\ldots,q\} : S \cap \mathscr{J}_h \neq \emptyset \,\}.$$

The subset S is feasible if the total weight of the selected items plus the setup weights of the activated classes does not exceed the knapsack capacity, namely

$$\sum_{j \in S} w_j \;+\; \sum_{h \in \mathscr{H}(S)} s_h \;\leq\; c.$$

Accordingly, the KPS can be equivalently stated as the problem of selecting a subset of items so as to maximize the total revenue of the selected items minus the setup costs of their activated classes.

Given a KPS instance, the KPS can be formally stated as:

$$\tilde{S} \in \arg\max_{S \subseteq \mathscr{I}} \left\{ \sum_{j \in S} p_j \;-\; \sum_{h \in \mathscr{H}(S)} f_h \;:\; \sum_{j \in S} w_j \;+\; \sum_{h \in \mathscr{H}(S)} s_h \leq c \right\}. \tag{2.11}$$

Example 2.8: KPS instance

We consider a knapsack of capacity $c = 20$ and a set $\mathscr{I} = \{1,2,3,4,5,6\}$ of 6 items ($n = 6$). The items are partitioned into $q = 2$ classes: $\mathscr{J}_1 = \{1,2,3\}$ and $\mathscr{J}_2 = \{4,5,6\}$. Each class has a setup cost and a setup weight: $f_1 = 3$, $s_1 = 2$ for class 1, and $f_2 = 4$, $s_2 = 3$ for class 2. The item revenues and weights are given as follows:

Items		$j=1$	$j=2$	$j=3$	$j=4$	$j=5$	$j=6$
Revenues	p_j	6	8	5	7	9	4
Weights	w_j	4	5	3	6	7	2

2.8.1 Assumptions

- We make the following assumptions:

  1. Each class contains at least one item. Formally:

  $$\mathscr{J}_h \neq \emptyset \qquad \forall\, h \in \{1,2,\ldots,q\}.$$

  2. Each item can fit into the knapsack together with the setup weight of its class (otherwise, it can never be selected). Formally:

  $$w_j + s_h \leq c \qquad \forall\, h \in \{1,2,\ldots,q\},\ \forall\, j \in \mathscr{J}_h.$$

2.9 The variants at a glance

- The table lists, for the instance of each section, the number of candidate solutions, the number of feasible solutions and the optimal value, computed by enumeration. In the BKP and in the UKP a candidate solution is the vector of the numbers of copies of every item, so that identical copies are not counted twice.
- The optimal solution is not always unique: the MKP, the MCKP and the KPS have several optimal solutions with the same value, and in the SSP three subsets reach the target.

Variant	Candidate solutions	Number	Feasible	Optimum	Optimal solutions
KP	subsets of 6 items	64	13	15	$\{1,4\}$
SSP	subsets of 6 items	64	3	yes	$\{6\}$, $\{1,4\}$, $\{2,3\}$
BKP	copies $x_j \leq b_j$	144	18	18	two copies of item 1 and item 5
UKP	copies $x_j \leq \hat{b}_j$	320	22	24	four copies of item 1
MDKP	subsets of 4 items	16	9	14	$\{3,4\}$
MKP	each item in a knapsack or none	243	109	30	$(S_1,S_2) = (\{1,2\},\{4,5\})$, $(\{2,4\},\{1,5\})$, $(\{4,5\},\{1,2\})$
MCKP	one item per class	8	3	17	$\{1,4,5\}$, $\{2,3,5\}$
KPS	subsets of 6 items	64	40	16	$\{1,2,3\}$, $\{4,5,6\}$, $\{1,2,3,6\}$

- In the MCKP only 3 of the 8 choices of one item per class fit in the knapsack: the feasibility condition of the assumptions is not a formality.

2.10 Remarks

- Dantzig (1957) sorted the items by non-increasing efficiency p_j/w_j and used this order to solve the continuous version of the KP; the same order returns in the bounds of the following chapters.
- Karp (1972) put the subset sum problem, called knapsack in his paper, among the first 21 NP-complete problems. The KP contains the SSP as the special case $p_j = w_j$, so it is NP-hard.
- Ibarra and Kim (1975) gave the first fully polynomial-time approximation scheme for the KP (Chapter «Approximation algorithms for the knapsack problem»).

2.11 Typical mistakes

Warning

- **Confusing the KP with the SSP.** The SSP asks for a total weight *exactly* equal to c and has a yes/no answer; it is the KP with $p_j = w_j$, where the answer is yes if the optimal value is c.
- **Stopping at the order of efficiency.** Taking the items in the order $1, 2, 3, \ldots$ of non-increasing p_j/w_j gives $\{1, 2\}$ with profit 11, while the optimal solution of Example 2.1 is $\{1, 4\}$ with profit 15.
- **Counting identical copies as different solutions.** In the BKP and in the UKP the copies (j, k) of the same item are interchangeable: different subsets of the extended set of items can give the same packing.
- **Choosing at most one item per class.** In the MCKP exactly one item per class is selected, so an instance can be infeasible; in the KPS a class with no selected item costs nothing.
- **Forgetting the assumptions.** An item with $w_j > c$ can never be selected, and if all the items fit the problem is trivial: both cases are removed before solving.

2.12 In summary

Summary

- The KP selects a subset of items of weight at most c and maximum profit; a solution is a subset, and 2^n subsets are candidates.
- The SSP, the BKP, the UKP, the MDKP, the MKP, the MCKP and the KPS change the number of copies, of resources, of knapsacks or the structure of the items; each one has its assumptions on the data.
- A solution found by eye is a primal bound; the cardinality bound is a dual bound that uses only the weights and the profits.
- The order of efficiency is not enough to find an optimal solution, even on a small instance.

Dashboard of the reference instance.

KP, Example 2.1 (max)	lower bound	upper bound	from
this chapter	15 (by eye)	17 (cardinality bound)	definition

Chapter 3

Bin packing and cutting stock

Prerequisites

- From this module, the chapter «Combinatorial optimization problems»: instances, feasible solutions, primal and dual bounds, the gap, and the Bell numbers that count the partitions of a set.
- From the chapter «Knapsack problems»: the form of the definitions and of the assumptions on the data.

Objectives

At the end of this chapter you will be able to:

- state the definition of the bin packing problem and of the cutting stock problem, with instances and feasible solutions;
- explain the assumptions on the data;
- count the candidate solutions of a small instance and recognize that they are partitions;
- find a feasible solution by hand and compute the weight bound;
- use the two bounds to certify that a solution is optimal.

3.1 The bin packing problem

Definition 3.1: Bin Packing Problem

Given:

1. an infinite number of bins, each with *capacity* $c \in \mathbb{Z}_{\geq 1}$,
2. a set $\mathscr{I} = \{1, 2, \ldots, n\}$ of n items,
3. for each item $j \in \mathscr{I}$, a *weight* $w_j \in \mathbb{Z}_{\geq 1}$,

the **Bin Packing Problem** (BPP) asks to decide in which bin to place each item, so that the total weight of the items in any bin does not exceed the bin capacity, and the total number of bins used is minimized.

- A **BPP instance** is specified by:
  1. two integer numbers:
     the bin capacity $c \in \mathbb{Z}_{\geq 1}$,
     the number of items $n \in \mathbb{Z}_{\geq 1}$,
  2. one vector of integer numbers of length n:
     $\boldsymbol{w} \in \mathbb{Z}^n_{\geq 1}$, containing the item weights.

- A **BPP feasible solution** is a family of **packing patterns**, i.e., subsets of items each assigned to one bin,
$$\mathcal{P} \subseteq 2^{\mathscr{I}}$$
such that:

  (i) $\mathcal{P}$ forms a partition of $\mathscr{I}$ into *nonempty* packing patterns, namely
$$\begin{gathered} P \neq \emptyset, \quad \text{for all } P \in \mathcal{P}, \\ P \cap P' = \emptyset, \quad \text{for all } P, P' \in \mathcal{P} \text{ with } P \neq P', \\ \bigcup_{P \in \mathcal{P}} P = \mathscr{I}; \end{gathered}$$

  (ii) the total weight of every packing pattern in $\mathcal{P}$ respects the bin capacity, namely
$$\sum_{j \in P} w_j \leq c, \quad \text{for all } P \in \mathcal{P}.$$

  Accordingly, the BPP can be equivalently stated as the problem of determining a partition of the item set into packing patterns such that the total weight of the items in each packing pattern does not exceed the bin capacity, and the number of packing patterns (bins) is minimized.

Given a BPP instance, the BPP can be formally stated as:

$$\tilde{\mathcal{P}} \in \arg\min_{\mathcal{P} \subseteq 2^{\mathscr{I}}} \left\{ |\mathcal{P}| \ : \begin{array}{l} P \neq \emptyset, \quad \forall P \in \mathcal{P}, \\ P \cap P' = \emptyset, \quad \forall P, P' \in \mathcal{P},\ P \neq P', \\ \bigcup_{P \in \mathcal{P}} P = \mathscr{I}, \\ \sum_{j \in P} w_j \leq c, \quad \forall P \in \mathcal{P} \end{array} \right\}. \tag{3.1}$$

Example 3.1: BPP instances

We consider a bin of capacity $c = 10$ and a set $\mathscr{I} = \{1, 2, 3, 4, 5\}$ of 5 items ($n = 5$), with the following item weights:

Items		$j=1$	$j=2$	$j=3$	$j=4$	$j=5$
Weights	w_j	6	5	4	7	3

We call this BPP instance Example 3.1: it is the reference instance of the chapter.

3.1.1 Assumptions

- We make the following assumptions:

  1. The total weight of all items exceeds the bin capacity (otherwise, all items could fit in a single bin). Formally:
  $$\sum_{j \in \mathscr{I}} w_j > c$$
  2. Each item individually fits into a single bin (otherwise, it cannot be packed at all). Formally:
  $$w_j \leq c, \quad \forall j \in \mathscr{I}$$

3.1.2 Counting the solutions, a first solution and a simple bound

- **Candidate solutions.** The candidate solutions are the partitions of $\mathscr{I}$: they are counted by the Bell numbers (Chapter «Combinatorial optimization problems»). For Example 3.1 we have $B(5) = 52$ partitions.
- **Feasible solutions.** A partition is feasible if every bin weighs at most 10. Enumeration gives 11 feasible partitions: 1 with 5 bins (one item per bin), 6 with 4 bins and 4 with 3 bins. Two bins are not enough, since the total weight 25 exceeds $2 \cdot 10$.
- **A first solution, by eye.** We pair the heavy items with the light ones: item 4 (weight 7) with item 5 (weight 3), and item 1 (weight 6) with item 3 (weight 4); item 2 (weight 5) stays alone. The solution

$$\mathcal{P} = \{\, \{4,5\},\ \{1,3\},\ \{2\} \,\}$$

uses 3 bins, of weights 10, 10 and 5: it is a **primal bound**, $\mathrm{UB} = 3 \geq z(I)$.

Observation 3.2: weight bound

Every feasible solution uses at least

$$\left\lceil \frac{\sum_{j \in \mathscr{I}} w_j}{c} \right\rceil$$

bins, so this value is a dual bound.

Proof. If a solution uses k bins, the total weight it carries is at most $k\,c$. The total weight of the items is $\sum_j w_j$, hence $k\,c \geq \sum_j w_j$. The number k is an integer, so $k \geq \lceil \sum_j w_j / c \rceil$. □

- **The dual bound and the gap.** In Example 3.1 we have $\sum_j w_j = 25$ and $c = 10$, so $\mathrm{LB} = \lceil 25/10 \rceil = 3$. The two bounds coincide: the gap is zero, the solution found by eye is optimal and $z(I) = 3$, without enumerating the partitions.
- The optimal solution is not unique: there are 4 optimal partitions,

$$\{\{1\},\{2,3\},\{4,5\}\}, \quad \{\{2\},\{1,3\},\{4,5\}\},$$
$$\{\{4\},\{1,5\},\{2,3\}\}, \quad \{\{4\},\{1,3\},\{2,5\}\}.$$

3.2 The cutting stock problem

Definition 3.3: Cutting Stock Problem

Given:

1. an infinite number of bins, each with *capacity* $c \in \mathbb{Z}_{\geq 1}$,
2. a set $\mathscr{I} = \{1, 2, \ldots, n\}$ of n items,
3. for each item $j \in \mathscr{I}$, a *weight* $w_j \in \mathbb{Z}_{\geq 1}$ and a *number of required copies* $d_j \in \mathbb{Z}_{\geq 1}$,

the **Cutting Stock Problem** (CSP) asks to decide in which bin to place each copy of every item, so that the total weight of all item copies in any bin does not exceed the bin capacity, the required number of copies of each item is placed into the bins, and the total number of bins used is minimized.

- A **CSP instance** is specified by:
  1. two integer numbers:
     the bin capacity $c \in \mathbb{Z}_{\geq 1}$,
     the number of items $n \in \mathbb{Z}_{\geq 1}$,
  2. two vectors of integer numbers of length n:
     $\boldsymbol{w} \in \mathbb{Z}^n_{\geq 1}$, containing the item weights,
     $\boldsymbol{d} \in \mathbb{Z}^n_{\geq 1}$, containing the required number of copies for each item.
- The problem is called the *Cutting Stock Problem* because, in its classical application, the bins correspond to large stock rolls (or sheets of material) that must be cut in order to produce the required copies of the items, also known as item demands.
- Given a CSP instance, we define the **extended set of items** as

$$\hat{\mathscr{I}} = \big\{ (j,k) : j \in \mathscr{I},\ k \in \{1, 2, \ldots, d_j\} \big\}, \tag{3.2}$$

  where each pair (j,k) represents the k-th required copy of item j. The cardinality of the extended set of items is

$$|\hat{\mathscr{I}}| = \sum_{j \in \mathscr{I}} d_j.$$

- A **CSP feasible solution** is a family of **packing patterns**, i.e., subsets of item copies each assigned to one bin,

$$\mathcal{P} \subseteq 2^{\hat{\mathscr{I}}}$$

  such that:

  (i) $\mathcal{P}$ forms a partition of $\hat{\mathscr{I}}$ into *nonempty* packing patterns, namely

$$P \neq \emptyset, \quad \text{for all } P \in \mathcal{P},$$
$$P \cap P' = \emptyset, \quad \text{for all } P, P' \in \mathcal{P} \text{ with } P \neq P',$$
$$\bigcup_{P \in \mathcal{P}} P = \hat{\mathscr{I}};$$

(ii) the total weight of the item copies in every packing pattern does not exceed the bin capacity, namely

$$\sum_{(j,k)\in P} w_j \leq c, \quad \text{for all } P \in \mathcal{P}.$$

Accordingly, the CSP can be equivalently stated as the problem of determining a partition of the extended item set into packing patterns such that the total weight in each packing pattern does not exceed the bin capacity, and the number of packing patterns (bins) is minimized.

Given a CSP instance, the CSP can be formally stated as:

$$\tilde{\mathcal{P}} \in \arg\min_{\mathcal{P}\subseteq 2^{\hat{\mathscr{I}}}} \left\{ |\mathcal{P}| \; : \; \begin{array}{l} P \neq \emptyset, \quad \forall\, P \in \mathcal{P}, \\ P \cap P' = \emptyset, \quad \forall\, P, P' \in \mathcal{P},\ P \neq P', \\ \bigcup_{P\in\mathcal{P}} P = \hat{\mathscr{I}}, \\ \sum_{(j,k)\in P} w_j \leq c, \quad \forall\, P \in \mathcal{P} \end{array} \right\}. \tag{3.3}$$

Example 3.2: CSP instances

We consider bins of capacity $c = 10$ and a set $\mathscr{I} = \{1, 2, 3, 4\}$ of 4 items ($n = 4$), with the following weights and required copies:

Items		$j=1$	$j=2$	$j=3$	$j=4$
Weights	w_j	4	5	3	6
Copies	d_j	2	1	3	1

3.2.1 Assumptions

- We make the following assumptions:

1. The total weight of all required item copies exceeds the bin capacity (otherwise, all copies could fit in a single bin). Formally:

$$\sum_{j\in\mathscr{I}} d_j\, w_j \;>\; c.$$

2. Each item individually fits into a single bin (otherwise, no copy of that item can be packed at all). Formally:

$$w_j \leq c, \qquad \forall\, j \in \mathscr{I}.$$

3.2.2 Counting the solutions, a first solution and a simple bound

- **Candidate solutions.** The instance of Example 3.2 has $2+1+3+1=7$ copies. The partitions of the extended set of items are counted by $B(7)=877$, where identical copies are counted as different items; 262 of them are feasible.
- **A first solution, by eye.** We pair item 4 (weight 6) with a copy of item 1 (weight 4), put the other copy of item 1 with two copies of item 3 ($4+3+3$), and the last copy of item 3 with item 2 ($3+5$):

$$\mathcal{P} = \big\{\, \{(4,1),(1,1)\},\ \{(1,2),(3,1),(3,2)\},\ \{(2,1),(3,3)\} \,\big\}$$

  uses 3 bins, of weights 10, 10 and 8, hence UB $= 3$.
- **The dual bound and the gap.** The weight bound of the bin packing problem holds with the weights of all the copies: LB $= \lceil \sum_j d_j w_j / c \rceil = \lceil 28/10 \rceil = 3$. The gap is zero, so the solution is optimal and $z(I) = 3$.

3.3 Remarks

- The worst-case behaviour of simple packing rules, such as First Fit, was analysed by Garey, Graham and Ullman (1972) and by Johnson, Demers, Ullman, Garey and Graham (1974).
- Johnson et al. (1974) proved that First Fit uses at most $\frac{17}{10}$ of the optimal number of bins, plus 2 (Chapter «Approximation algorithms for bin packing»).

3.4 Typical mistakes

Warning

- **Believing that the weight bound is always tight.** With three items of weight 6 and $c = 10$ the bound gives $\lceil 18/10 \rceil = 2$, but no two items fit together, so 3 bins are needed.
- **Counting the bins by the weights alone.** The weight bound ignores how the items fit together; it is a dual bound, and the primal bound still needs a solution.
- **Listing the same packing twice.** In the cutting stock problem the copies of an item are interchangeable: partitions that differ only by the exchange of identical copies are the same packing.
- **Ignoring the order of the bins.** A solution is a partition: $\{\{4,5\},\{1,3\},\{2\}\}$ and $\{\{2\},\{4,5\},\{1,3\}\}$ are the same solution.
- **Forgetting the assumptions.** An item with $w_j > c$ fits in no bin, and if the total weight is at most c one bin is enough.

3.5 In summary

Summary

- The BPP partitions the items into the minimum number of bins of capacity c; the cutting stock problem is the BPP with several identical copies of every item, and the BPP is the case $d_j = 1$.
- A solution is a partition, so the candidates are counted by the Bell numbers and grow very fast.
- A solution found by eye is a primal bound; the weight bound $\lceil \sum_j w_j / c \rceil$ is a dual bound.
- When the two bounds coincide, the gap is zero and the solution is certified optimal.

Dashboard of the reference instances.

instance (min)	lower bound	upper bound	from
BPP, Example 3.1	3 (weight bound)	3 (by eye)	this chapter
CSP, Example 3.2	3 (weight bound)	3 (by eye)	this chapter

Chapter 4

Vertex coloring

Prerequisites

- From this module, the chapter «Combinatorial optimization problems»: instances, feasible solutions, primal and dual bounds, the gap, and the Bell numbers that count the partitions of a set.
- From the chapter «Bin packing and cutting stock problems»: the weight bound as an example of a dual bound.
- No knowledge of graph theory is needed: the definitions used here are recalled in a short section.

Objectives

At the end of this chapter you will be able to:

- use the basic notions on graphs: neighbourhood, degree, induced graph, complement graph, clique and stable set;
- state the definition of the vertex coloring problem and the assumptions on the graph;
- count the candidate solutions of a small instance and recognize that they are partitions;
- find a coloring by hand and compute the clique bound and the stable set bound;
- explain why the clique bound is not always tight.

4.1 A short review of graphs

Definition 4.1: graph, neighbourhood, induced graph and complement graph

- A **graph** $G = (V, E)$ is given by a finite set V of **vertices** and a set E of **edges**, each edge being a 2-element subset of V. We write $n = |V|$ and $m = |E|$; the vertices are labelled $1, 2, \ldots, n$. All the graphs of this chapter are simple and undirected.
- Two distinct vertices u and v are **adjacent** if $\{u, v\} \in E$. The **neighbourhood** of a vertex v is the set $N(v) = \{\, u \in V : \{u, v\} \in E \,\}$, and its **degree** is $\deg(v) = |N(v)|$.
- For a subset $U \subseteq V$ the **induced graph** $G[U] = (U, E[U])$ has the vertices of U and the edges of E with both endpoints in U.
- The **complement graph** $\overline{G} = (V, \overline{E})$ has the same vertices as G, and two distinct vertices are adjacent in $\overline{G}$ if and only if they are not adjacent in G.

Definition 4.2: clique and stable set

Let $G = (V, E)$ be a graph.

1. A **clique** is a subset $K \subseteq V$ of pairwise adjacent vertices: $\{u, v\} \in E$ for all $u, v \in K$ with $u \neq v$.
2. A **stable set** (also called **independent set**) is a subset $Q \subseteq V$ of pairwise non-adjacent vertices: $\{u, v\} \notin E$ for all $u, v \in Q$ with $u \neq v$.

The **clique number** $\omega(G)$ is the largest number of vertices of a clique, and the **stability number** $\alpha(G)$ is the largest number of vertices of a stable set.

- A stable set of G is a clique of the complement graph $\overline{G}$, and vice versa. A vertex alone and the empty set are both a clique and a stable set.

A clique is a group of vertices that pairwise *conflict*; a stable set is a group of vertices with no conflict. In a coloring, the vertices of a clique need different colors, and the vertices of a stable set can share one.

4.2 The vertex coloring problem

Definition 4.3: Vertex Coloring Problem

Given:

1. a simple *graph* $G = (V, E)$, where

$$V = \{1, 2, \ldots, n\}$$

is the set of n *vertices*, and

$$E \subseteq \big\{\, \{u, v\} : u, v \in V,\ u \neq v \,\big\}$$

is the set of m *edges*,

the **Vertex Coloring Problem** (VCP) asks to decide how to assign a color to each vertex, so that no two adjacent vertices share the same color, and the total number of colors used is minimized.

- A **VCP instance** is specified by:
  1. two integer numbers:
     - the number of vertices $n \in \mathbb{Z}_{\geq 1}$,
     - the number of edges $m \in \mathbb{Z}_{\geq 1}$,
  2. two vectors of integer numbers of length m:
     - $\boldsymbol{u} \in \mathbb{Z}^m_{\geq 1}$, containing the first endpoint of the edges,
     - $\boldsymbol{v} \in \mathbb{Z}^m_{\geq 1}$, containing the second endpoint of the edges.
- A **VCP feasible solution** is a family of **color classes**, i.e., subsets of vertices each assigned to one color,

$$\mathcal{Q} \subseteq 2^V$$

such that:

(i) $\mathcal{Q}$ forms a partition of V into *nonempty* color classes, namely

$$\begin{gathered} Q \neq \emptyset, \quad \text{for all } Q \in \mathcal{Q}, \\ Q \cap Q' = \emptyset, \quad \text{for all } Q, Q' \in \mathcal{Q} \text{ with } Q \neq Q', \\ \bigcup_{Q \in \mathcal{Q}} Q = V; \end{gathered}$$

(ii) each color class in $\mathcal{Q}$ is an *stable set*, namely

$$\{u, v\} \notin E, \quad \text{for all } Q \in \mathcal{Q}, \text{ for all } u, v \in Q \text{ with } u \neq v.$$

Accordingly, the VCP can be equivalently stated as the problem of determining a partition of the vertex set into color classes such that each color class is an stable set (i.e., its vertices can receive the same color), and the number of color classes (colors) is minimized.

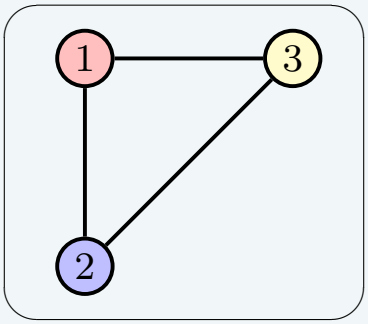
Given a VCP instance, the VCP can be formally stated as:

$$\tilde{\mathcal{Q}} \in \arg\min_{\mathcal{Q} \subseteq 2^V} \left\{ |\mathcal{Q}| \; : \; \begin{array}{l} Q \neq \emptyset, \quad \forall Q \in \mathcal{Q}, \\ Q \cap Q' = \emptyset, \quad \forall Q, Q' \in \mathcal{Q} \text{ with } Q \neq Q', \\ \bigcup_{Q \in \mathcal{Q}} Q = V, \\ \{u, v\} \notin E, \quad \forall Q \in \mathcal{Q}, \; \forall u, v \in Q \text{ with } u \neq v \end{array} \right\}. \tag{4.1}$$

- The optimal value of the VCP is the **chromatic number** $\chi(G)$ of the graph: the minimum number of colors (stable sets) needed to color all the vertices. The color classes of a solution are stable sets, and the number of classes is the number of colors used.

Example 4.1: VCP instance: a triangle

We consider the following graph $G = (V, E)$ with 3 vertices ($n = 3$) and 3 edges ($m = 3$):



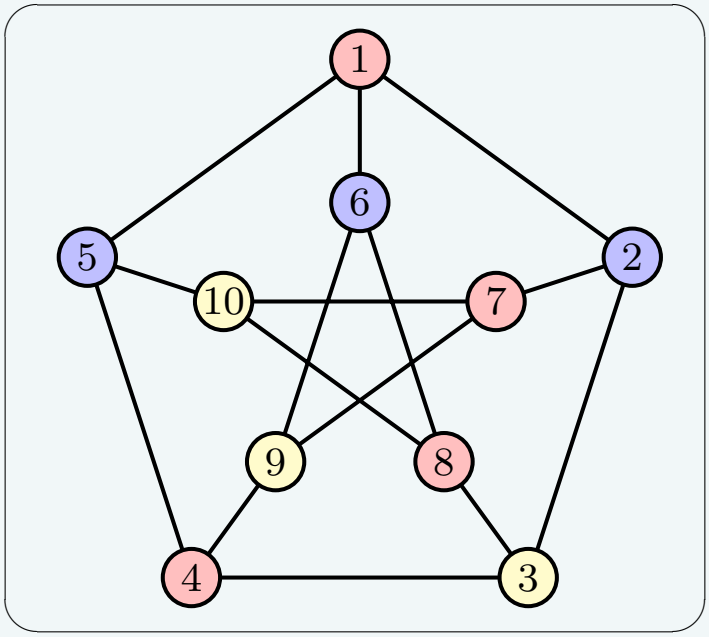
The vertex set V and the edge set E are:

$$V = \{\ 1,\ 2,\ 3\ \}, \qquad E = \{\ \{1,2\},\ \{1,3\},\ \{2,3\}\ \}.$$

We call this VCP instance Example 4.1.

Example 4.2: VCP instance: the Petersen graph

We consider now the following graph $G = (V, E)$ with 10 vertices ($n = 10$) and 15 edges ($m = 15$):



The vertex set V and the edge set E are:

$$V = \{\ 1,\ 2,\ 3,\ 4,\ 5,\ 6,\ 7,\ 8,\ 9,\ 10\ \},$$

$$\begin{aligned} E = \{\ & \{1,2\},\ \{1,5\},\ \{1,6\},\ \{2,3\},\ \{2,7\},\\ & \{3,4\},\ \{3,8\},\ \{4,5\},\ \{4,9\},\ \{5,10\},\\ & \{6,8\},\ \{6,9\},\ \{7,9\},\ \{7,10\},\ \{8,10\}\ \}. \end{aligned}$$

We call this VCP instance Example 4.2: it is the reference instance of the chapter.

4.2.1 Assumptions

- We make the following assumptions:

  1. The graph has no isolated vertices. An isolated vertex can receive any color used by the rest of the graph; if the graph has no edges, the optimal value is one. Formally:

  $$\deg(v) \geq 1, \qquad \forall\, v \in V.$$

  2. No vertex has its neighbourhood contained in the neighbourhood of another vertex. Formally:

  $$N(u) \not\subseteq N(v), \qquad \forall\, u, v \in V,\ u \neq v.$$

- The second assumption can be made without loss of generality, because:

  1. if $N(u) \subseteq N(v)$, the vertex u can be removed without changing the chromatic number: every coloring of $G \setminus \{u\}$ extends to u with the color of v;
  2. the vertices u and v are not adjacent: otherwise $v \in N(u) \subseteq N(v)$, which is impossible in a simple graph, so u and v can share a color.

4.2.2 Two simple bounds

Observation 4.4: clique bound

For every graph $G = (V, E)$ we have

$$\omega(G) \leq \chi(G).$$

Proof. Since all vertices in any clique are pairwise adjacent, they must all receive different colors in any VCP feasible solution. Thus, at least $\omega(G)$ different colors are needed to color the vertices of any maximum clique, implying $\omega(G) \leq \chi(G)$. □

Observation 4.5: stable set bound

For every graph $G = (V, E)$ we have

$$\left\lceil \frac{n}{\alpha(G)} \right\rceil \leq \chi(G).$$

Proof. In a feasible solution with k color classes, every class is a stable set, so it has at most $\alpha(G)$ vertices. The n vertices are covered by the classes, hence $n \leq k\,\alpha(G)$. The number k is an integer, so $k \geq \lceil n/\alpha(G) \rceil$. □

- The clique bound is useful when the graph has a large clique, and the stable set bound when its stable sets are small. Both are dual bounds: a clique and a stable set are found without knowing $\chi(G)$.

4.2.3 Counting the solutions, a first solution and the bounds

- **Candidate solutions.** The candidate solutions are the partitions of V, counted by the Bell numbers (Chapter «Combinatorial optimization problems»). For Example 4.1 we have $B(3) = 5$ partitions and only one is feasible: three singletons, since every pair of vertices is adjacent. Hence $\chi(G) = 3$.
- For Example 4.2 we have $B(10) = 115\,975$ partitions, and 7 430 of them are feasible, that is, have only stable sets as classes.
- **A first solution, by eye.** The coloring shown in Example 4.2 uses 3 colors:

  $$\mathcal{Q} = \big\{\, \{1, 4, 7, 8\},\ \{2, 5, 6\},\ \{3, 9, 10\} \,\big\}.$$

  No edge joins two vertices of the same class, so it is feasible: it is a **primal bound**, UB $= 3 \geq \chi(G)$.
- **Dual bounds on the Petersen graph.** The graph has no triangle, so $\omega(G) = 2$ and the clique bound gives only LB $= 2$: a gap of 1 remains.
- The vertices $1, 2, 3, 4, 5$ form a cycle with 5 vertices, and so do $6, 7, 8, 9, 10$ (the edges of the figure). A stable set contains at most 2 vertices of each cycle, so $\alpha(G) \leq 4$; the set $\{1, 4, 7, 8\}$ shows that $\alpha(G) = 4$.

- The stable set bound gives $\lceil 10/4 \rceil = 3$. The two bounds coincide: the gap is zero and $\chi(G) = 3$.

4.3 Remarks

- Brooks (1941) proved that a connected graph that is neither complete nor an odd cycle can be colored with at most Δ colors, where Δ is the largest degree.
- Karp (1972) included the chromatic number among his 21 NP-complete problems. Garey, Johnson and Stockmeyer (1976) proved that deciding whether 3 colors are enough remains NP-complete on planar graphs of maximum degree 4.

4.4 Typical mistakes

Warning

- **Confusing a clique with a stable set.** In a clique all the pairs are edges and the vertices need different colors; in a stable set no pair is an edge and the vertices can share a color.
- **Believing that the clique bound is tight.** It is not: the Petersen graph has $\omega(G) = 2$ and $\chi(G) = 3$, and the same happens for a cycle with 5 vertices.
- **Counting the permutations of the colors as different solutions.** A solution is a partition of the vertices into stable sets: exchanging the names of two colors gives the same solution.
- **Using a class that is not a stable set.** A color class with two adjacent vertices makes the solution infeasible, whatever the number of colors.
- **Forgetting the assumptions.** An isolated vertex and a vertex with $N(u) \subseteq N(v)$ can be removed before solving, without changing the chromatic number.

4.5 In summary

Summary

- The VCP partitions the vertices of a graph into the minimum number of stable sets; the optimal value is the chromatic number $\chi(G)$.
- A clique needs different colors and a stable set can share one; the cliques of G are the stable sets of the complement graph.
- A coloring found by eye is a primal bound; the clique bound $\omega(G)$ and the stable set bound $\lceil n/\alpha(G) \rceil$ are dual bounds.
- The bounds do not always meet: on the Petersen graph the clique bound leaves a gap, the stable set bound closes it.

Dashboard of the reference instances.

instance (min)	lower bound	upper bound	from
Example 4.1 (triangle)	3 (clique bound)	3 (by eye)	this chapter
Example 4.2 (Petersen)	3 (stable set bound)	3 (by eye)	this chapter

Chapter 5

The travelling salesman problem

Prerequisites

- From this module, the chapter «Combinatorial optimization problems»: what a combinatorial optimization problem is, how the number of solutions explodes, and the notions of primal bound, dual bound and gap.
- From the same chapter, the reference TSP instance with 5 cities, which this chapter studies in detail.

Objectives

At the end of this chapter you will be able to:

- state the TSP, the asymmetric TSP and the metric TSP, and describe an instance and a tour;
- compute the length of a tour, and count the tours of an instance;
- explain why listing all the tours is hopeless already for 20 cities;
- build a first tour by hand with the nearest neighbour rule;
- compute a lower bound on the optimal length from the two shortest edges of every city, and measure the gap with a tour.

5.1 The travelling salesman problem

Definition 5.1: Travelling salesman problem

Given:

1. a set $\mathscr{V} = \{1, 2, \dots, n\}$ of n *cities*,
2. for each pair of distinct cities $i, j \in \mathscr{V}$, a *distance* $d_{ij} \in \mathbb{Z}_{\geq 1}$, with $d_{ij} = d_{ji}$,

the **travelling salesman problem** (TSP) asks to find a closed route that starts from a city, visits every other city exactly once and returns to the starting city, so that the total distance travelled is minimized.

- A **TSP instance** is specified by:
  1. one integer number:
     the number of cities $n \in \mathbb{Z}_{\geq 3}$,
  2. one symmetric matrix of integer numbers of size $n \times n$:
     $D = (d_{ij})$, containing the distances; the diagonal entries are not used.

- A **TSP feasible solution**, called a **tour**, is a permutation of the cities

$$\sigma = (\sigma_1, \sigma_2, \dots, \sigma_n),$$

read as the closed route $\sigma_1 \to \sigma_2 \to \cdots \to \sigma_n \to \sigma_1$. Its **length** is

$$L(\sigma) = \sum_{k=1}^{n-1} d_{\sigma_k \sigma_{k+1}} + d_{\sigma_n \sigma_1},$$

where the last term is the distance travelled to return to the starting city.

- Different permutations may describe the same tour: the starting city can be any of the n cities, and, since the distances are symmetric, the tour can be travelled in either direction.

Given a TSP instance, the TSP can be formally stated as:

$$\tilde{\sigma} \in \arg\min_{\sigma} \left\{ L(\sigma) = \sum_{k=1}^{n-1} d_{\sigma_k \sigma_{k+1}} + d_{\sigma_n \sigma_1} \ : \ \sigma \text{ is a permutation of } \mathscr{V} \right\}. \tag{5.1}$$

- We denote by $z(I)$ the length of an optimal tour $\tilde{\sigma}$ of the instance I.

5.1.1 Assumptions

- We make the following assumptions:

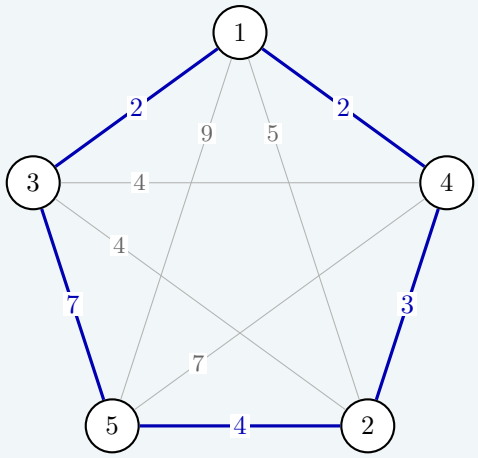
  1. There are at least three cities (with one or two cities there is only one route). Formally:
  $$n \geq 3$$
  2. The distances are symmetric (travelling from i to j costs as much as travelling from j to i). Formally:
  $$d_{ij} = d_{ji}, \quad \forall i, j \in \mathcal{V},\ i \neq j$$

Example 5.1: TSP instance

We consider a set $\mathcal{V} = \{1, 2, 3, 4, 5\}$ of 5 cities ($n = 5$), with the following distances:

d_{ij}	1	2	3	4	5
1	—	5	2	2	9
2	5	—	4	3	4
3	2	4	—	4	7
4	2	3	4	—	7
5	9	4	7	7	—

This is the TSP instance of the MIP modelling course. The instance can be drawn as a complete graph, with one node per city and one edge, labelled with the distance, per pair of cities. The bold edges form the tour $\sigma = (1, 3, 5, 2, 4)$, and the cities are drawn in the order of the tour:



Its length is $L(\sigma) = d_{13} + d_{35} + d_{52} + d_{24} + d_{41} = 2 + 7 + 4 + 3 + 2 = 18$.

5.2 Variants: metric and asymmetric TSP

- Several heuristic algorithms for the TSP behave well, and some of them have a performance guarantee, only if the distances satisfy the triangle inequality: going directly from i to j is never longer than going through a third city.

Definition 5.2: Metric TSP

A TSP instance is **metric** if its distances satisfy the **triangle inequality**

$$d_{ij} \leq d_{ik} + d_{kj} \qquad \text{for all distinct } i, j, k \in \mathcal{V}.$$

- The instance of Example 5.1 is metric: all the $5 \cdot 4 \cdot 3 = 60$ triangle inequalities hold, some of them with equality, e.g., $d_{15} = 9 = d_{12} + d_{25} = 5 + 4$.
- Instances obtained from points in the plane, with d_{ij} equal to the Euclidean distance between the points, are metric (when the distances are not rounded).
- In the **asymmetric travelling salesman problem** (ATSP) the distances are not required to be symmetric: d_{ij} may differ from d_{ji}, as when travelling along one-way streets or uphill and downhill.
- The definition of tour and of its length $L(\sigma)$ is unchanged, but the direction now matters: a tour and the same tour travelled backwards are different solutions.
- The TSP is the special case of the ATSP in which $d_{ij} = d_{ji}$ for all i, j.

5.3 Counting the tours

Proposition 5.3

A symmetric TSP instance with $n \geq 3$ cities has

$$\frac{n!}{2n} = \frac{(n-1)!}{2}$$

distinct tours; an ATSP instance has $\frac{n!}{n} = (n-1)!$ distinct tours.

- There are $n!$ permutations of the cities. Each tour is described by n permutations, one for every possible starting city, so there are $n!/n = (n-1)!$ cyclic orderings: this is the ATSP count.
- In the symmetric case the tour travelled backwards has the same length and is not a new solution: every tour is counted twice, and we divide by 2.

Example 5.2: enumeration of the tours

The instance of Example 5.1 has $\frac{(5-1)!}{2} = 12$ distinct tours, which can be enumerated:

tour	L	tour	L	tour	L
$(1,3,5,2,4)$	18	$(1,3,4,2,5)$	22	$(1,2,3,5,4)$	25
$(1,3,2,5,4)$	19	$(1,4,3,2,5)$	23	$(1,3,2,4,5)$	25
$(1,2,5,3,4)$	22	$(1,2,4,5,3)$	24	$(1,4,2,3,5)$	25
$(1,2,5,4,3)$	22			$(1,2,4,3,5)$	28
				$(1,2,3,4,5)$	29

Hence the tour $\sigma = (1,3,5,2,4)$ is the unique optimal tour, and $z(I) = 18$. The same instance, read as an ATSP instance, would have $4! = 24$ tours.

- The number of tours grows faster than any exponential function:

n	5	10	15	20
$\frac{(n-1)!}{2}$	12	181 440	$\approx 4.4 \cdot 10^{10}$	$\approx 6.1 \cdot 10^{16}$

- A computer that evaluates 10^9 tours per second needs about $6.1 \cdot 10^7$ seconds, that is about two years, to list the tours of 20 cities. Enumeration is therefore useful only as a check on tiny instances, and the algorithms of this course must do better.

5.4 A first tour by hand

- The simplest way to build a tour is the **nearest neighbour rule**: start from a city and, as long as there are unvisited cities, move to the closest unvisited one; at the end, return to the starting city. In case of a tie we take the city with the smallest index.

Example 5.3: nearest neighbour from city 1

On the instance of Example 5.1, starting from city 1:

1. from 1 the closest cities are 3 and 4, both at distance 2: we go to 3;
2. from 3 the unvisited cities 2 and 4 are both at distance 4: we go to 2;
3. from 2 the closest unvisited city is 4, at distance 3; then only 5 is left, at distance $d_{45} = 7$;
4. we return from 5 to 1 at distance $d_{51} = 9$.

The tour is $\sigma = (1,3,2,4,5)$, of length $2 + 4 + 3 + 7 + 9 = 25$.

- The last edges of the nearest neighbour tour are forced, and may be very long: here the final edge $\{5,1\}$ costs 9. The tour of length 25 is worse than the optimal one, of length 18, by 7.
- From the nearest neighbour tour alone we cannot tell how far we are from the optimum: we need a lower bound.

5.5 A combinatorial lower bound

- For a city i, let $a_i \le b_i$ be the two smallest distances from i to the other cities.

Observation 5.4

For every symmetric TSP instance, the optimal length satisfies

$$z(I) \;\ge\; \left\lceil \frac{1}{2} \sum_{i \in V} (a_i + b_i) \right\rceil .$$

- In a tour, every city is entered by one edge and left by another one, so it has exactly two edges of the tour, each at least as long as a_i and b_i respectively.
- Summing over all the cities, every edge of the tour is counted twice, once for each of its end cities: hence $2L(\sigma) \ge \sum_i (a_i + b_i)$ for every tour σ.
- The distances are integer, so is $L(\sigma)$, and we can round the bound up.

Example 5.4: the bound on the reference instance

For the instance of Example 5.1 the two smallest distances of each city are:

city i	1	2	3	4	5	sum
a_i	2	3	2	2	4	
b_i	2	4	4	3	7	
$a_i + b_i$	4	7	6	5	11	33

Hence $z(I) \ge \lceil 33/2 \rceil = 17$. Together with the nearest neighbour tour we obtain

$$17 \le z(I) \le 25,$$

an absolute gap of 8, that is a relative gap of $\frac{8}{17} \approx 47.1\%$. The tour $(1, 3, 5, 2, 4)$ of length 18 closes most of it, and only the enumeration of the 12 tours proves that 18 is optimal.

5.6 Remarks

- The largest instance solved to optimality, pla85900, has 85 900 points from a chip design problem. It was solved in 2005–2006 with the Concorde code; the optimal tour, of length 142 382 641, was certified in 2009 by Applegate, Bixby, Chvátal, Cook, Espinoza, Goycoolea and Helsgaun.
- On the road maps of real countries, the record of 2025 is the instance korea81998, with 81 998 bars of South Korea.
- The TSP is strongly NP-hard (chapter «Combinatorial optimization problems»). The approximation algorithm of Christofides (1976) is in the chapter «$P\|C_{\max}$, metric TSP and the limits of approximation».

5.7 Typical mistakes

Warning

- **Counting $n!$ tours.** The starting city does not change the tour, and in the symmetric case neither does the direction: the number of tours is $(n-1)!/2$.
- **Forgetting the return edge.** The length of a tour includes the last distance $d_{\sigma_n \sigma_1}$, from the last city back to the first one.
- **Taking two cycles for a tour.** Two disjoint cycles that together visit every city once are not a tour: a tour is a single cycle through all the cities.
- **Reading the bound as a tour.** The value 17 of the two-shortest-edges bound is not the length of a tour; it only says that no tour is shorter.
- **Using the triangle inequality without checking it.** The guarantees of the heuristic algorithms hold only for metric instances.

5.8 In summary

Summary

- The TSP asks for a shortest closed route through all the cities; a solution is a cyclic ordering of the cities, called a tour.
- A symmetric instance with n cities has $(n-1)!/2$ tours, an asymmetric one $(n-1)!$; listing them is possible only for very small n.
- The nearest neighbour rule gives a tour quickly, with no guarantee; the two shortest edges of every city give a lower bound on the optimal length.
- The metric TSP, with the triangle inequality, is the case where approximation guarantees exist.

Dashboard of the TSP instance (minimization: the lower bound is a dual bound, the upper bound is the length of a tour):

Chapter	Lower bound	Upper bound	From
This chapter	17	25	two shortest edges; nearest neighbour from city 1
This chapter, enumeration	18	18	the 12 tours

Chapter 6

Parallel machine scheduling

Prerequisites

- From this module, the chapter «Combinatorial optimization problems»: what a combinatorial optimization problem is, the number of candidate solutions, and primal and dual bounds.
- From the same chapter, the reference $P \,||\, C_{\max}$ instance with 7 jobs and 3 machines, and the partitions of a set (Bell and Stirling numbers).

Objectives

At the end of this chapter you will be able to:

- classify a scheduling problem with the three-field notation, and state the problem $P \,||\, C_{\max}$;
- describe a schedule by the sets of jobs of the machines, and compute loads and makespan;
- count the assignments and the distinct schedules of an instance;
- build a first schedule by hand, assigning every job to the least loaded machine;
- compute a lower bound on the optimal makespan and use it to prove that a schedule is optimal.

6.1 Scheduling on identical parallel machines

Definition 6.1: Scheduling on identical parallel machines

Given:

1. a set of m identical *machines*,
2. a set $\mathscr{J} = \{1, 2, \ldots, n\}$ of n *jobs*,
3. for each job $j \in \mathscr{J}$, a *processing time* $t_j \in \mathbb{Z}_{\geq 1}$,

the problem $P \,||\, C_{\max}$ asks to assign each job to a machine, so that the time needed to complete all the jobs, i.e., the largest total processing time assigned to a machine, is minimized.

- The name $P \,||\, C_{\max}$ comes from the three-field notation of scheduling problems (Section 6.2).
- A $P \,||\, C_{\max}$ **instance** is specified by:
  1. two integer numbers:
     the number of machines $m \in \mathbb{Z}_{\geq 1}$,
     the number of jobs $n \in \mathbb{Z}_{\geq 1}$,
  2. one vector of integer numbers of length n:
     $\boldsymbol{t} \in \mathbb{Z}^n_{\geq 1}$, containing the processing times.

- A $P \,||\, C_{\max}$ **feasible solution** is an assignment of the jobs to the machines, which we store in a vector $\text{mach} = (\text{mach}_1, \text{mach}_2, \ldots, \text{mach}_n)$:

  $$\text{mach}_j = h \quad \Longleftrightarrow \quad \text{job } j \text{ is assigned to machine } h.$$

  The set of jobs assigned to machine h is $J_h = \{j \in \mathscr{J} : \text{mach}_j = h\}$, and its **load** is

  $$L_h = \sum_{j \in J_h} t_j.$$

  Each machine processes its jobs one after the other, without idle times, so machine h completes its jobs at time L_h. The **makespan** of the solution is the completion time of the last job,

  $$C_{\max} = \max_{h \in \{1,2,\ldots,m\}} L_h.$$

- The order in which a machine processes its jobs does not change its load: a solution is fully described by the sets $J_1, J_2, \ldots, J_m$, which form a partition of $\mathscr{J}$ (some of them may be empty).

Given a $P\,||\,C_{\max}$ instance, the problem can be formally stated as:

$$\min \left\{ \max_{h\in\{1,\dots,m\}} \sum_{j\in J_h} t_j \;:\; J_1, J_2, \dots, J_m \text{ is a partition of } \mathscr{J} \right\}. \tag{6.1}$$

- We denote by $z(I)$ the optimal makespan of the instance I.

6.1.1 Assumptions

- We make the following assumption:

  1. There are more jobs than machines (otherwise each job gets a machine of its own, and the optimal makespan is $\max_j t_j$). Formally:

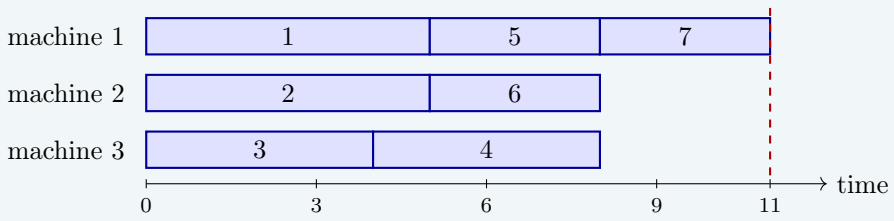
$$n > m$$

Example 6.1: $P\,||\,C_{\max}$ instance

We consider $m = 3$ machines and a set $\mathscr{J} = \{1, 2, \dots, 7\}$ of 7 jobs ($n = 7$), with the following processing times:

	$j=1$	$j=2$	$j=3$	$j=4$	$j=5$	$j=6$	$j=7$
t_j	5	5	4	4	3	3	3

This is the $P\,||\,C_{\max}$ instance of the MIP modelling course. The assignment mach $= (1, 2, 3, 3, 1, 2, 1)$ gives $J_1 = \{1, 5, 7\}$, $J_2 = \{2, 6\}$, $J_3 = \{3, 4\}$, with loads $L_1 = 11$, $L_2 = 8$, $L_3 = 8$ and makespan 11:



6.2 The three-field notation

- Scheduling problems are classified with the **three-field notation** $\alpha \,|\, \beta \,|\, \gamma$ of Graham, Lawler, Lenstra and Rinnooy Kan, where:
  1. α describes the **machine environment**: 1 for a single machine, P for identical parallel machines, Q for parallel machines with different speeds, R for unrelated parallel machines;
  2. β describes the **job characteristics**, such as release dates r_j or precedence constraints *prec*; an empty field means that there are none;
  3. γ describes the **objective function**, such as the makespan $C_{\max}$ or the total completion time $\sum_j C_j$.
- In this notation, $P \,||\, C_{\max}$ is the problem of scheduling jobs without additional characteristics on identical parallel machines, minimizing the makespan.

6.3 Counting the solutions

- Every job can be assigned to any of the m machines, independently of the others: there are m^n assignments mach.
- The machines are identical, so two assignments that differ only by the names of the machines describe the same schedule. The distinct schedules are the partitions of $\mathscr{J}$ into at most m non-empty sets, and their number is

$$\sum_{k=1}^{m} S(n,k),$$

  where $S(n,k)$ is the Stirling number of the second kind, the number of partitions of n elements into exactly k non-empty sets.

Example 6.2: counting the schedules

For the instance of Example 6.1 there are $3^7 = 2187$ assignments. The Stirling numbers are $S(7,1) = 1$, $S(7,2) = 63$ and $S(7,3) = 301$, so the distinct schedules are

$$1 + 63 + 301 = 365.$$

By enumeration, the schedules with the smallest makespan, 9, are exactly two:

$$\{1,3\},\{2,4\},\{5,6,7\} \qquad \text{and} \qquad \{1,4\},\{2,3\},\{5,6,7\},$$

which give $3! \cdot 2 = 12$ of the 2187 assignments. The optimal schedule is therefore not unique.

- The growth is exponential in n: with 3 machines, 20 jobs have $3^{20} \approx 3.5 \cdot 10^9$ assignments, and 30 jobs have $3^{30} \approx 2.1 \cdot 10^{14}$.

6.4 A first schedule by hand

- A simple way to build a schedule is to take the jobs one by one, in the order $1, 2, \ldots, n$, and to assign each job to the machine with the smallest load at that moment; in case of a tie we take the machine with the smallest index.

Example 6.3: assigning every job to the least loaded machine

On the instance of Example 6.1 the loads of the three machines evolve as follows:

job j (t_j)	machine chosen	loads (L_1, L_2, L_3)
1 (5)	1	$(5, 0, 0)$
2 (5)	2	$(5, 5, 0)$
3 (4)	3	$(5, 5, 4)$
4 (4)	3	$(5, 5, 8)$
5 (3)	1	$(8, 5, 8)$
6 (3)	2	$(8, 8, 8)$
7 (3)	1	$(11, 8, 8)$

We obtain the assignment mach $= (1, 2, 3, 3, 1, 2, 1)$ of the example, with makespan 11.

- When the last job arrives all the machines have load 8: whichever machine receives it, the largest load becomes $8 + 3 = 11$.
- We do not know yet whether 11 is far from the optimal makespan: we need a lower bound.

6.5 A combinatorial lower bound

Observation 6.2

For every $P \,||\, C_{\max}$ instance, the optimal makespan $z(I)$ satisfies

$$z(I) \;\geq\; \max\left\{ \max_{j \in \mathscr{J}} t_j,\;\; \left\lceil \frac{\sum_{j \in \mathscr{J}} t_j}{m} \right\rceil \right\}.$$

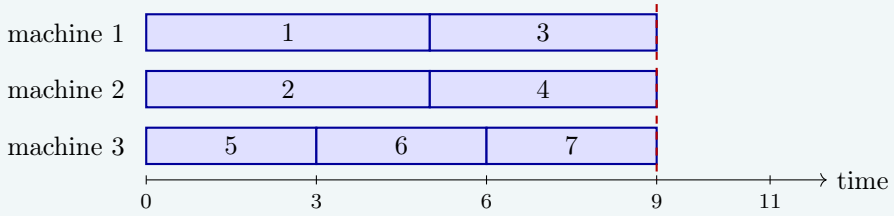
- The machine processing the longest job has load at least $\max_j t_j$.
- The m loads sum up to $\sum_j t_j$, so the largest one is at least their average $\frac{1}{m}\sum_j t_j$, and, being an integer, at least its ceiling.

Example 6.4: the bound on the reference instance

On the instance of Example 6.1 the bound is $\max\{5, \lceil 27/3 \rceil\} = 9$. Together with the schedule of the previous section,

$$9 \leq z(I) \leq 11.$$

The assignment mach $= (1, 2, 1, 2, 3, 3, 3)$ gives $J_1 = \{1, 3\}$, $J_2 = \{2, 4\}$, $J_3 = \{5, 6, 7\}$, all with load 9, and makespan 9:



It attains the lower bound, so it is optimal: $z(I) = 9$. The three machines finish together, and no idle time is left before the makespan.

- The bound is not always attained. With $m = 2$ machines and three jobs of processing times $(4, 3, 3)$ it gives $\max\{4, \lceil 10/2 \rceil\} = 5$, but the best schedule is $\{1\}, \{2, 3\}$, with makespan 6.

6.6 Relation with the bin packing problem

- Given a value T, a schedule with makespan at most T exists if and only if the jobs, seen as items of weight t_j, can be packed into m bins of capacity T: the machines are the bins and the loads are the total weights of the packing patterns.
- The two problems fix and minimize opposite quantities:

	number of bins / machines	capacity / makespan
BPP	minimized	fixed (c)
$P \,\|\|\, C_{\max}$	fixed (m)	minimized

- For this reason the two problems share many ideas: several heuristic algorithms for one of them have a counterpart for the other.

6.7 Remarks

- In 1966 R. L. Graham proved that any list of jobs, processed greedily, gives a makespan at most $2 - 1/m$ times the optimal one.
- In 1969 he proved the better ratio $4/3 - 1/(3m)$ for the list sorted by non-increasing processing time (the rule called LPT). Both guarantees are proved in the chapter «$P\|C_{\max}$, metric TSP and the limits of approximation».

6.8 Typical mistakes

Warning

- **Counting m^n different schedules.** The machines are identical: renaming them gives the same schedule, so the distinct schedules are the partitions of the jobs into at most m sets.
- **Confusing makespan and total load.** The makespan is the largest load of a machine, not the sum of the processing times and not the average load.
- **Forgetting the ceiling.** The processing times are integer, so is the makespan: the average load $27/3$ is rounded up when it is not an integer.
- **Assuming that the bound is attained.** The bound of the previous section is a dual bound: a schedule with that makespan may not exist, as the instance $(4, 3, 3)$ on two machines shows.
- **Believing that the optimal schedule is unique.** The instance of Example 6.1 has two optimal schedules, and twelve optimal assignments.

6.9 In summary

Summary

- The problem $P\,||\,C_{\max}$ asks to assign jobs to identical machines minimizing the largest load, the makespan; a solution is a partition of the jobs into at most m sets.
- There are m^n assignments, but only $\sum_{k=1}^{m} S(n,k)$ distinct schedules.
- Giving each job to the least loaded machine builds a schedule quickly; the maximum between the longest job and the rounded average load is a lower bound on the optimal makespan.
- When a schedule attains the lower bound it is optimal; otherwise the gap remains open.
- The problem is the mirror of bin packing: here the number of machines is fixed and the capacity is minimized.

Dashboard of the $P\,||\,C_{\max}$ instance (minimization: the lower bound is a dual bound, the upper bound is the makespan of a schedule):

Chapter	Lower bound	Upper bound	From
This chapter	9	11	longest job and average load; least loaded machine
This chapter, improved schedule	9	9	$J_1 = \{1, 3\}$, $J_2 = \{2, 4\}$, $J_3 = \{5, 6, 7\}$

Chapter 7

Capacitated lot sizing

Prerequisites

- From this module, the chapter «Combinatorial optimization problems»: what a combinatorial optimization problem is, the number of candidate solutions, and primal and dual bounds.

Objectives

At the end of this chapter you will be able to:

- describe a production plan by its quantities, stocks and setups, and compute its cost;
- state the capacitated lot sizing problem and name its main variants;
- decide whether a plan is feasible, and count the feasible setup choices of an instance;
- build a first plan by hand, merging the demands of consecutive periods;
- compute a lower bound on the cost from the number of setups that the total demand requires.

7.1 The capacitated lot sizing problem

Definition 7.1: Capacitated lot sizing problem

Given:

1. a set $T = \{1, 2, \dots, m\}$ of m *time periods*,
2. for each period $t \in T$, a *demand* $d_t \in \mathbb{Z}_{\geq 1}$ and a *capacity* $cap_t \in \mathbb{Z}_{\geq 1}$, both counted in units of one product,
3. for each period $t \in T$, three non-negative integer costs: the *setup cost* sc_t, the *variable production cost* vc_t per unit produced and the *holding cost* hc_t per unit kept in stock at the end of the period,

the **capacitated lot sizing problem** (CLS) asks to decide how much to produce in every period, so that every demand is satisfied with what is produced and what is in

stock, the capacities are respected, and the total cost of setups, production and holding is minimized.

- A **CLS instance** is specified by:

  1. one integer number:

     the number of periods $m \in \mathbb{Z}_{\geq 1}$,

  2. five vectors of integer numbers of length m:

     $\boldsymbol{d},\ \boldsymbol{cap} \in \mathbb{Z}^m_{\geq 1}$, containing demands and capacities,

     $\boldsymbol{sc},\ \boldsymbol{vc},\ \boldsymbol{hc} \in \mathbb{Z}^m_{\geq 0}$, containing the costs.

- A **CLS feasible solution** is a **production plan**, that is a vector of integer quantities

$$\boldsymbol{x} = (x_1, x_2, \ldots, x_m) \in \mathbb{Z}^m_{\geq 0},$$

  where x_t is the quantity produced in period t. The stock left at the end of period t is

$$s_t(\boldsymbol{x}) = \sum_{u=1}^{t} (x_u - d_u),$$

  because the stock of period $t-1$ plus the production of period t must cover the demand of period t, and what remains is the stock of period t. The plan is **feasible** if

$$x_t \leq cap_t \quad \text{and} \quad s_t(\boldsymbol{x}) \geq 0 \qquad \forall t \in T, \qquad s_m(\boldsymbol{x}) = 0.$$

- There is a **setup** in period t when $x_t > 0$: we write $y_t = 1$ in this case and $y_t = 0$ otherwise. The **cost** of a plan is

$$c(\boldsymbol{x}) = \sum_{t \in T} \big(sc_t\, y_t + vc_t\, x_t + hc_t\, s_t(\boldsymbol{x}) \big).$$

Given a CLS instance, the CLS can be formally stated as:

$$\tilde{\boldsymbol{x}} \in \arg\min_{\boldsymbol{x}} \left\{ c(\boldsymbol{x}) \ :\ x_t \leq cap_t,\ s_t(\boldsymbol{x}) \geq 0,\ \forall t \in T,\ s_m(\boldsymbol{x}) = 0,\ \boldsymbol{x} \in \mathbb{Z}^m_{\geq 0} \right\}. \tag{7.1}$$

- We denote by $z(I)$ the cost of an optimal plan $\tilde{\boldsymbol{x}}$ of the instance I.

7.1.1 Assumptions

- We make the following assumptions:

  1. The stock is empty at the beginning and at the end of the horizon. Leaving stock at the end only adds cost, so no optimal plan needs it; an initial stock is a variant of the problem (next section).
  2. The capacities allow to satisfy the demands, period after period (otherwise there is no feasible plan). Formally:

$$\sum_{u=1}^{t} cap_u \geq \sum_{u=1}^{t} d_u, \qquad \forall t \in T$$

  3. The quantities are integer. With integer data, once the periods with a setup are fixed, what remains is a minimum cost flow problem, which has an integer optimal solution: restricting the quantities to integers does not change the optimal cost.

7.2 Variants

- In the **uncapacitated lot sizing problem** the capacities are not binding, for example $cap_t \geq \sum_u d_u$. It is solved in polynomial time by dynamic programming.
- With **several products**, a set P of products shares one resource: d_{pt} is the demand of product $p \in P$ in period t, and cap_t is the time available in period t. Producing a unit of p takes vt_{pt} units of time, and a setup of p costs sc_{pt} and takes st_{pt} units of time.
- With an **initial stock**, the quantity si_p of product p available at the beginning of the horizon is a decision, bought at the cost fc_p per unit.
- With **overtime**, the capacity is not fixed: working time in normal hours costs cw_t per unit up to cap_t^N, and overtime costs co_t per unit up to cap_t^O.
- With **unsatisfied demand**, a part of the demand may be left unsatisfied, at a penalty cu_{pt} per unit.
- The models of all these variants, with their big-M constraints, are the subject of the chapter «Formulations of capacitated lot sizing». This chapter studies the problem of the definition above.

7.3 The reference instance

Example 7.1: CLS instance

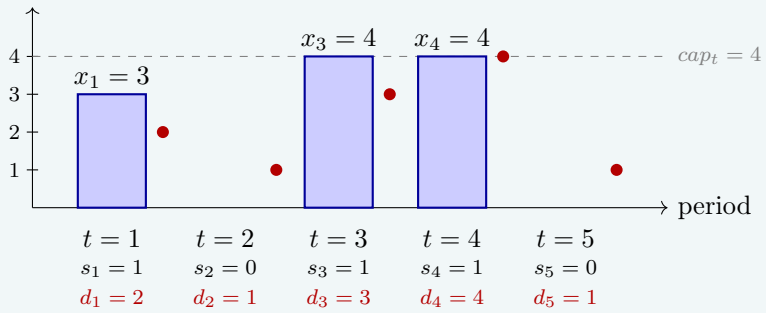
We consider $m = 5$ periods, with the following data, all integer:

period t	1	2	3	4	5
demand d_t	2	1	3	4	1
capacity cap_t	4	4	4	4	4
setup cost sc_t	50	50	50	50	50
production cost vc_t	0	0	0	0	0
holding cost hc_t	10	10	10	10	10

This is the lot sizing instance of the course on MIP heuristics, with the demands counted in units of ten items. The total demand is $2 + 1 + 3 + 4 + 1 = 11$.
The plan $\boldsymbol{x} = (3, 0, 4, 4, 0)$ is feasible: the stocks are $s = (1, 0, 1, 1, 0)$, all non-negative, with $s_5 = 0$, and every quantity is at most 4. There are 3 setups, in the periods 1, 3 and 4:

$$c(\boldsymbol{x}) = 3 \cdot 50 + 10 \cdot (1 + 0 + 1 + 1 + 0) = 150 + 30 = 180.$$

The plan is drawn below: the bars are the quantities produced, the dots the demands.



7.4 Counting the solutions

- The combinatorial part of the decision is the choice of the periods with a setup: there are 2^m subsets of periods. Not all of them are feasible, because the periods with a setup must have enough capacity to produce everything.
- For a given choice of the setup periods there may be many production plans, since the quantities can be distributed in many ways: the number of feasible plans is larger than the number of feasible setup choices.

Example 7.2: counting the plans

For the instance of Example 7.1 there are $2^5 = 32$ subsets of periods. Let k_t be the number of setups in the periods $1, \dots, t$: the cumulative demands $2, 3, 6, 10, 11$ require, with capacity 4, at least one setup in the periods 1–2, two in the periods 1–3 and three in the periods 1–4. Since $d_1 > 0$ there is a setup in period 1, and in the periods $2, 3, 4$ we need at least

two setups, one of which in the periods 2 or 3:

$$(y_2, y_3, y_4) \in \{(1,1,0), (1,0,1), (0,1,1), (1,1,1)\}.$$

The period 5 is free, so the feasible setup choices are $4 \cdot 2 = 8$ out of 32.
By enumeration, these setup choices carry 94 feasible plans. The costs of six of them are:

plan	cost	plan	cost	plan	cost
$(3,0,4,4,0)$	**180**	$(4,0,3,4,0)$	200	$(2,1,4,4,0)$	220
$(3,0,3,4,1)$	210	$(4,0,4,3,0)$	210	$(2,1,3,4,1)$	250

The plan $(3,0,4,4,0)$ is the unique optimal plan, and $z(I) = 180$.

- Without the capacities the same instance has $2^{m-1} = 16$ plans that produce only when the stock is empty, and the optimal cost is 170: here the capacity costs 10.
- The optimal plan $(3,0,4,4,0)$ produces in period 4 with a stock of 1 unit, so the property of Wagner and Whitin does not hold when the capacities are binding.
- The number of setup choices grows as 2^m: with $m = 52$ weeks it is more than $4 \cdot 10^{15}$.

7.5 A first plan by hand

- The simplest feasible plan is **lot-for-lot**: in every period we produce exactly the demand of that period. There is no stock, and there are m setups; it is feasible when every demand fits in the capacity of its period.
- A plan is improved by **merging**: the demand of a period is produced in an earlier period, which saves a setup and costs the stock of the units anticipated.

Example 7.3: from lot-for-lot to a better plan

On the instance of Example 7.1:

1. the lot-for-lot plan $(2,1,3,4,1)$ has 5 setups and no stock: cost $5 \cdot 50 = 250$;
2. producing the demand of period 2 in period 1 saves one setup (-50) and keeps 1 unit for one period $(+10)$: the plan $(3,0,3,4,1)$ costs 210;
3. period 4 cannot produce the demand of period 5 as well, because $4 + 1 > cap_4 = 4$; but period 3 can anticipate one unit of period 4, so that period 4 has room for the demand of period 5: the plan $(3,0,4,4,0)$ saves a setup (-50) and keeps two more units $(+20)$.

The cost is $210 - 50 + 20 = 180$.

- We have found the plan of cost 180 by hand, but we cannot yet tell whether a cheaper plan exists: we need a lower bound.

7.6 A combinatorial lower bound

- Let $D = \sum_{t \in T} d_t$ be the total demand. Let k be the smallest number of periods whose capacities sum up to at least D, which is found by taking the periods with the largest capacities.

Observation 7.2

For every CLS instance, every feasible plan has at least k setups, and

$$z(I) \;\geq\; \text{the sum of the } k \text{ smallest setup costs.}$$

- The plan must produce D units in the periods with a setup, and a period without setup produces nothing: the capacities of the periods with a setup sum up to at least D, hence they are at least k.
- The costs of production and holding are non-negative: dropping them, and paying only k setups, the cheapest ones, gives a bound on the cost of every plan.

Example 7.4: the bound on the reference instance

For the instance of Example 7.1 we have $D = 11$ and all capacities equal to 4, so $k = \lceil 11/4 \rceil = 3$. All the setup costs are 50, hence

$$z(I) \geq 3 \cdot 50 = 150.$$

Together with the plans of the previous section we obtain

$$150 \leq z(I) \leq 180,$$

an absolute gap of 30, that is a relative gap of $\frac{30}{150} = 20\%$. The enumeration of the 94 plans proves that 180 is optimal.

7.7 Remarks

- In 1958 Wagner and Whitin gave a dynamic programming algorithm for the case without capacity and with demand that varies over time. It rests on a property of the optimal plans: one produces only when the stock is empty (chapter «Dynamic programming for lot sizing»).
- With capacities the property fails, and the problem is NP-hard (chapter «Combinatorial optimization problems»).

7.8 Typical mistakes

Warning

- **Producing late.** The production of a period cannot serve the demand of an earlier period: the stock $s_t(\boldsymbol{x})$ must be non-negative in every period.
- **Charging a setup when nothing is produced.** A setup cost is paid only in the periods with $x_t > 0$.
- **Forgetting the stock of the last periods.** The holding cost is paid on the stock at the end of every period, including the periods in which the plan produces nothing.
- **Believing that one produces only with empty stock.** The property of Wagner and Whitin holds without capacities; with capacities the optimal plan of Example 7.1 violates it.
- **Counting 2^m feasible plans.** The 2^m subsets of periods are the setup choices: some are infeasible, and each feasible one carries one or more plans.

7.9 In summary

Summary

- The capacitated lot sizing problem asks for the production quantities of every period, with capacities, setups and stock, at the minimum total cost; a solution is a production plan.
- The combinatorial part is the choice of the periods with a setup: there are 2^m choices, only some of them feasible, and each carries one or more plans.
- Lot-for-lot gives a plan quickly, merging the demands improves it; the number of setups that the total demand requires gives a lower bound.
- With capacities, optimal plans may produce while the stock is positive; without capacities they never do.
- The variants add products, setup times, initial stock, overtime or unsatisfied demand; their models are in the chapter on formulations.

Dashboard of the CLS instance (minimization: the lower bound is a dual bound, the upper bound is the cost of a plan):

Chapter	Lower bound	Upper bound	From
This chapter	150	250	three setups at least; lot-for-lot
This chapter, merging	150	180	the plan $(3, 0, 4, 4, 0)$
This chapter, enumeration	180	180	the 94 feasible plans